

claude-windows / d4b5afb5-2f03-4b58-8952-573f1381d166

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4b5afb5-2f03-4b58-8952-573f1381d166\subagents\agent-abdb69c58b2d38715.jsonl`  
- SHA-256: `c2bb0a7c0063ea484bcf51f02ccc326543e4e92124373d03b075c221f789863f`  
- Source modified: `2026-06-21T07:42:46+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `d4b5afb5-2f03-4b58-8952-573f1381d166`
```

Transcript

1. user (2026-06-21T07:39:51.496Z)

You are mapping the integration surface for a specific end-to-end goal so an engineer can plan it. Be precise and ground every claim in real file:line evidence ? do NOT speculate.

THE GOAL (owner's words): "stitch up the UI with the tool so the tool can drive it; do a dry run with a VIDEO ON A GCS BUCKET, operating via the UI, and get the resulting highlight reel DOWNLOADED to the local machine. Then repeat with the UI in Hindi to confirm the multilingual workflow works."

So the target CUJ: open the web UI ? point at a video in a GCS bucket ? the engine processes it (rally detection ? highlights) ? review/select rallies ? compile ? download the reel locally. All driven from the UI.

TWO REPOS:

```
- **SPA (the UI):** `C:/Users/avidu/Projects/khelsutra/web/` (Vite + React + TS). Read  
`src/api/client.ts`, `src/api/types.ts`, `src/App.tsx`, and the components. Map EXACTLY what the  
SPA does today (it's "job-tethered": loads rallies for `?video=<id>` then  
query?select?compile?download) and what endpoints it calls.  
- **Engine (the tool):** `C:/Users/avidu/Projects/badminton-highlight-indexer/` (FastAPI +  
Python). Read `backend/main.py` (the API), `backend/api/uploads.py` (chunked upload),  
`backend/storage/` (StorageBackend ? esp. any GCS/bucket backend + a `gdrive` stub),  
`backend/worker/___main__.py` (the `python -m backend.worker {infer|train}` bucket pipeline), and  
the config. Map the API endpoints and what each does.
```

ALSO read the design docs for context: `khelsutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md` and
`khelsutra/docs/PER\_RALLY\_SELECTION.md`.

ANSWER THESE PRECISELY (with file:line evidence for each):

1. ****Engine API inventory**** ? list every relevant endpoint (`/api/process`, `/api/jobs/\*`, `/api/query`, `/api/compile`, `/api/highlights/\*`, `/api/health`, upload endpoints, any `/api/videos`, any reel-download endpoint). For each: method, what it does, and whether it's wired to a real detector or a stub.
2. ****Bucket ingest**** ? can the engine ingest/process a video that lives in a GCS bucket TODAY? Via what (the worker `backend.worker infer` storage pipeline? a `StorageBackend`? `/api/process` with a path?). Is GCS supported or only local/MinIO/gdrive-stub? Quote the storage backend code.
3. ****Detector / compute**** ? what detectors exist (Gemini cloud path vs native WASB GPU vs stub)? Which run on CPU (no GPU)? What does a real dry-run need (a Gemini API key? a GPU? WSL?)? Is there a CPU-only path that produces real-ish rallies?
4. ****Reel download to local**** ? how does a compiled reel get to the user's local machine? `/api/compile` returns a server-local filepath (not a URL) per earlier notes ? is there a `download\_url` / a `GET /api/highlights/{id}/{name}` streaming endpoint the browser can hit? (The engine "M1" reportedly added `download\_url`.) Quote it.
5. ****The SPA gap**** ? what does the SPA NOT do yet that this CUJ needs? (e.g., no ingest/process UI ? it assumes a processed job; does it call `/api/process`? does it let you pick a bucket video? how does the user get a `video\_id`?) Be specific about what UI/wiring is missing.
6. ****The CHEAPEST path to a dry run**** ? given what exists, what is the minimum to demonstrate

"bucket video ? UI ? download" end to end? (e.g., pre-process the bucket video with the worker CLI to get a `video\_id`+job, run the engine API, open the SPA `?video=<id>` against it, select?compile?download ? vs building full in-UI ingest.) List the concrete steps + which pieces already work vs must be built/run.

7. ****Dependencies / unknowns**** ? what must be decided or stood up (engine host: local Windows / GCP VM / the droplet; compute: Gemini-CPU vs GPU; which bucket video; CORS/edge-auth for the SPA?engine connection; is the engine even runnable locally on Windows or does it need WSL/Linux?).

Return a thorough markdown report structured by those 7 questions. Ground everything in file:line. Flag honestly where something is a stub, missing, or unverified.

2. assistant (2026-06-21T07:39:53.788Z)

I'll map the integration surface for this end-to-end goal. Let me start by pulling both repos fresh and reading the key files in parallel.

3. user (2026-06-21T07:39:57.100Z)

Already up to date.
---KHELSUTRA STATUS---

master...origin/master

?? .claude/
?? site/access.db

4. user (2026-06-21T07:39:59.123Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m1-deployment-profile' from the remote, but no such ref was fetched.
---ENGINE STATUS---

feat/serving-m1-deployment-profile...origin/feat/serving-m1-deployment-profile [gone]

?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---BRANCH---
feat/serving-m1-deployment-profile

5. assistant (2026-06-21T07:40:01.769Z)

Both repos pulled. The engine is on branch `feat/serving-m1-deployment-profile` (note: remote ref is gone ? the "M1" branch). Let me read the key files in parallel, starting with the SPA and the engine API.

6. user (2026-06-21T07:40:02.724Z)

```
1      // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2      // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3      // correlation id (header + logs) so a journey is traceable end to end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type { RallySegment, QueryResponse, CompileResponse, EngineConfig } from
"./types";
6
7      const BASE = "/api";
8
9      export class ApiError extends Error {
10         readonly cid?: string;
11         readonly status?: number;
12         readonly body?: string;
13         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
14             super(message);
```

```

15     this.name = "ApiError";
16     this.cid = opts.cid;
17     this.status = opts.status;
18     this.body = opts.body;
19     if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
20 }
21 }
22
23 interface RequestOpts {
24     method?: string;
25     body?: string;
26     headers?: Record<string, string>;
27     /** Correlation id to thread through this call; auto-generated if absent. */
28     cid?: string;
29 }
30
31 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
32     const cid = opts.cid ?? newCorrelationId();
33     const url = `${BASE}${path}`;
34     const method = opts.method ?? "GET";
35     log("info", "api.request", { cid, method, url });
36
37     let res: Response;
38     try {
39         res = await fetch(url, {
40             method,
41             body: opts.body,
42             headers: { "content-type": "application/json", "x-correlation-id": cid,
... (opts.headers ?? {}) },
43         });
44     } catch (err) {
45         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
46         log("error", "api.network_error", { cid, method, url, error: String(err) });
47         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
48     }
49
50     if (!res.ok) {
51         const body = await res.text().catch(() => "");
52         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
53         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
54     }
55
56     const data = (await res.json()) as T;
57     log("info", "api.response", { cid, method, url, status: res.status });
58     return data;
59 }
60
61 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
62 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
63     const data = await request<QueryResponse>("/query", {
64         method: "POST",
65         body: JSON.stringify({ video_id: videoId }),
66         cid,
67     });
68     return data.play_segments ?? [];
69 }
70
71 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
72 export async function compileReel(
73     videoId: string,

```

```

74     segmentIds: number[],
75     outputFilename: string,
76     cid?: string,
77 ): Promise<CompileResponse> {
78     return request<CompileResponse>("/compile", {
79         method: "POST",
80         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
81         cid,
82     });
83 }
84
85 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
86 export function highlightUrl(videoId: string, filename: string): string {
87     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
88 }
89
90 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
91 export async function getConfig(cid?: string): Promise<EngineConfig> {
92     return request<EngineConfig>("/config", { method: "GET", cid });
93 }
94
95 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
96 export function minShotCount(cfg: EngineConfig): number | undefined {
97     return cfg.stitching?.min_shot_count;
98 }
99

```

7. user (2026-06-21T07:40:03.209Z)

```

1     // Types mirror the engine's verified contract (badminton-highlight-indexer).
2     // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3     // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5     export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12    }
13
14    export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21        server: string | null;
22        /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23        receiver: string | null;
24        metadata: RallyMetadata;
25        /** motion-only fabricated sub-events ? DO NOT surface. */
26        events?: unknown[];
27    }
28
29    export interface QueryResponse {
30        play_segments: RallySegment[];

```

```

31     }
32
33     export interface CompileResponse {
34         status: string;
35         /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36         filepath: string;
37     }
38
39     export interface EngineConfig {
40         stitching?: {
41             /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42             */
43             min_shot_count?: number;
44             [key: string]: unknown;
45         };
46         [key: string]: unknown;
47     }

```

8. user (2026-06-21T07:40:03.670Z)

```

1     import { useEffect, useMemo, useState } from "react";
2     import { useTranslation, Trans } from "react-i18next";
3     import { useRallySelection } from "../hooks/useRallySelection";
4     import { RallyGrid } from "../components/RallyGrid";
5     import { QueryBar } from "../components/QueryBar";
6     import { SelectionFooter } from "../components/SelectionFooter";
7     import { LanguageSwitcher } from "../components/LanguageSwitcher";
8     import { CompileResult, type BuildState } from "../components/CompileResult";
9     import { runQuery, type ParsedQuery } from "../lib/query";
10    import { floorRisk } from "../lib/floor";
11    import { reelFilename } from "../lib/format";
12    import { log, newCorrelationId } from "../lib/log";
13    import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
14    from "../api/client";
15    import type { EngineConfig, RallySegment } from "../api/types";
16
17    /** One compact, structured description of how a query was understood (for the trace
18    log). */
19    function parseTrace(q: ParsedQuery) {
20        return {
21            raw: q.raw,
22            recognized: q.recognized,
23            filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
24            sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
25            limit: q.limit ?? null,
26            unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
27        };
28    }
29
30    // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
31    play_segments
32    // contract; only honest fields are surfaced. A real session loads from POST /api/query.
33    const DEMO_RALLIES: RallySegment[] = [
34        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
35        metadata: { shot_count: 9, shot_count_confidence: "High" } },
36        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
37        metadata: { shot_count: 5 } },
38        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
39        metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
40        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
41        metadata: {} },
42        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
43        metadata: { shot_count: 11 } },
44        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,

```

```

metadata: { shot_count: 24, shot_count_confidence: "High" } },
37     ];
38
39     type LoadState =
40         | { status: "demo" }
41         | { status: "loading" }
42         | { status: "loaded"; count: number }
43         | { status: "error"; error: string };
44
45     export default function App() {
46         const { t } = useTranslation();
47         // Job-tethered (D10): a finished job's video_id arrives via ?video=<id>; absent ?
demo data.
48         const videoId = useMemo(() => new
URLSearchParams(window.location.search).get("video"), []);
49
50         const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
51         const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
52         const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
53         const [reelName, setReelName] = useState("highlights");
54         const [build, setBuild] = useState<BuildState>({ status: "idle" });
55
56         // Display order is separate from selection: a query can re-sort the cards while the
shared
57         // selection set (sel) stays the single source of truth for what lands in the reel.
58         const sel = useRallySelection(rallies, "all");
59
60         // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
61         useEffect(() => {
62             if (!videoId) return;
63             const cid = newCorrelationId();
64             log("info", "rallies.load_start", { cid, video_id: videoId });
65             let alive = true;
66             listRallies(videoId, cid)
67                 .then((rs) => {
68                     if (!alive) return;
69                     setRallies(rs);
70                     setOrder(rs);
71                     sel.apply(rs.map((r) => r.id));
72                     setLoadState({ status: "loaded", count: rs.length });
73                     log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
74                 })
75                 .catch((err) => {
76                     if (!alive) return;
77                     setLoadState({ status: "error", error: String(err) });
78                     log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
});
79                 });
80         return () => {
81             alive = false;
82         };
83         // videoId is stable for the page; sel.apply is a stable callback.
84         // eslint-disable-next-line react-hooks/exhaustive-deps
85     }, [videoId]);
86
87     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
88     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
89     const [cfg, setCfg] = useState<EngineConfig | null>(null);
90     useEffect(() => {
91         let alive = true;
92         getConfig()
93             .then((c) => alive && setCfg(c))

```

```

94         .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
95     return () => {
96         alive = false;
97     };
98 }, []);
99 const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
100
101 // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
102 const ordinalOf = useMemo(() => {
103     const m = new Map<number, number>();
104     [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
105     return (id: number) => m.get(id) ?? 0;
106 }, [rallies]);
107
108 // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
109 const onBuild = async () => {
110     if (build.status === "building") return;
111     const cid = newCorrelationId();
112     const filename = reelFilename(reelName);
113     const ids = [...sel.selected];
114     if (!videoId) {
115         log("info", "build.no_video", { cid, selected: ids.length });
116         setBuild({
117             status: "error",
118             error: t("build.noVideo"),
119         });
120         return;
121     }
122     setBuild({ status: "building" });
123     log("info", "build.start", {
124         cid,
125         video_id: videoId,
126         filename,
127         selected: ids.length,
128         total_sec: Math.round(sel.totalSec),
129         below_floor: warning?.atRisk.length ?? 0,
130     });
131     try {
132         const res = await compileReel(videoId, ids, filename, cid);
133         const url = highlightUrl(videoId, filename);
134         setBuild({ status: "done", url, filename });
135         log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
136     } catch (err) {
137         const status = err instanceof ApiError ? err.status : undefined;
138         const detail = err instanceof ApiError && err.body ? parseDetail(err.body) :
String(err);
139         setBuild({ status: "error", error: detail });
140         log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });
141     }
142 };
143
144 // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
145 // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
146 // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
147 const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
148     const cid = newCorrelationId();
149     log("info", "query.parse", { cid, source, ...parseTrace(q) });
150

```

```

151      // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
152      // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
153      if (q.unavailable.length > 0) {
154          log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
155      }
156
157      if (!q.recognized) {
158          log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
159          return; // empty / unparseable ? leave the user's curation untouched
160      }
161
162      try {
163          const { ordered, selectedIds } = runQuery(rallies, q);
164          setOrder(ordered);
165          sel.apply(selectedIds);
166          log("info", "query.apply", {
167              cid,
168              source,
169              filters: q.filters.length,
170              sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
171              limit: q.limit ?? null,
172              total: rallies.length,
173              selected: selectedIds.length,
174              selected_ids: selectedIds,
175              dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
176          });
177          if (selectedIds.length === 0) {
178              // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
179              log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
180          }
181      } catch (err) {
182          // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
183          log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
184      }
185  };
186
187  // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
188  const onToggle = (id: number) => {
189      log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
190      sel.toggle(id);
191  };
192  const onBulk = (op: "select_all" | "clear" | "invert") => {
193      const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
194      log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
195      if (op === "select_all") sel.selectAll();
196      else if (op === "clear") sel.clearAll();
197      else sel.invert();
198  };
199
200  return (
201      <main className="min-h-screen bg-paper text-ink pb-24">
202          <header className="bg-ink text-white">
203              <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
204                  <span className="text-xl font-extrabold tracking-tight">
205                      Khel<span className="text-green">?</span>Sutra <span className="font-

```



```

semibold text-lime">Studio</span>
206         </span>
207         <LanguageSwitcher />
208     </div>
209 </header>
210
211     <section className="mx-auto max-w-5xl px-6 py-10">
212         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
213             {t("app.badge")}
214         </span>
215         <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
216         <p className="mt-2 max-w-2xl text-ink/70">
217             <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
218         </p>
219
220         <ModeBanner load={loadState} videoId={videoId} />
221
222         <div className="mt-6">
223             <QueryBar onApply={onApplyQuery} />
224         </div>
225
226         <div className="mt-6 flex flex-wrap gap-2 text-sm">
227             <button type="button" onClick={() => onBulk("select_all")} className="rounded-
full border border-black/15 px-3 py-1 hover:border-green">{t("bulk.selectAll")}</button>
228             <button type="button" onClick={() => onBulk("clear")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">{t("bulk.clear")}</button>
229             <button type="button" onClick={() => onBulk("invert")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">{t("bulk.invert")}</button>
230         </div>
231
232         <div className="mt-5">
233             <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
234         </div>
235
236         <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
237     </section>
238
239     <SelectionFooter
240         count={sel.count}
241         totalSec={sel.totalSec}
242         reelName={reelName}
243         onReelNameChange={setReelName}
244         onBuild={onBuild}
245         building={build.status === "building"}
246         warning={warning}
247     />
248 </main>
249 );
250 }
251
252 /** Pull FastAPI's {"detail": "..."} message out of an error body, else show it raw. */
253 function parseDetail(body: string): string {
254     try {
255         const j = JSON.parse(body);
256         if (j && typeof j.detail === "string") return j.detail;
257     } catch {
258         /* not JSON ? fall through */
259     }
260     return body;
261 }
262
263 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {

```

```

264     if (load.status === "demo") {
265         return (
266             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
267                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
268             </p>
269         );
270     }
271     if (load.status === "loading") {
272         return (
273             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
274                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
275             </p>
276         );
277     }
278     if (load.status === "error") {
279         return (
280             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
281             text-red-800" data-testid="mode-banner">
282                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
283             components={{ code: <code /> }} />
284             </p>
285         );
286     }
287     return (
288         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
289             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
290             <strong className="text-ink/70" /> }} />
291             </p>
292         );
293     }
294 }

```

9. user (2026-06-21T07:40:04.368Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
13    for direct user-facing messages).
14    logging.basicConfig(
15        level=logging.INFO,
16        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
17        datefmt='%H:%M:%S'
18    )
19    logger = logging.getLogger(__name__)
20    from fastapi import FastAPI, HTTPException
21    from fastapi.middleware.cors import CORSMiddleware
22    from pydantic import BaseModel
23    from typing import List, Dict, Any, Optional
24
25    from fastapi.staticfiles import StaticFiles
26    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
27    from backend.pipeline.validators.base import registry
28    from backend.infrastructure.database import Database
29    from backend.pipeline.segmenters.base import segmenter_registry
30    from backend.pipeline.stitcher.compiler import VideoCompiler

```

```

30     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32     from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
33     from backend.results import Failure # standardized error/result contract
34     from backend.reporting import emit_indexer_report
35
36     app = FastAPI(title="Sports Highlight Indexer API")
37
38     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
39     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
40     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
41     app.add_middleware(
42         CORSMiddleware,
43         allow_origins=["*"],
44         allow_credentials=False,
45         allow_methods=["*"],
46         allow_headers=["*"],
47     )
48
49     # Serves static frontend files directly (now under api/ per approved structure)
50     os.makedirs("backend/api/static", exist_ok=True)
51     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
52
53     @app.get("/", response_class=HTMLResponse)
54     def serve_index():
55         index_path = "backend/api/static/index.html"
56         if os.path.exists(index_path):
57             with open(index_path, "r", encoding="utf-8") as f:
58                 return f.read()
59         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
60
61     # Global variables initialized at startup
62     db_instance: Optional[Database] = None
63     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
64
65     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
66     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
67     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
68     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
69     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
70     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
71     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
72     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
73         JobWaiting,
74         _arm_wake_timer,
75         _drain_due_jobs,
76         _get_executor,
77         _job_to_request,
78         _MAX_DRAIN_WAKE_SEC,
79         _run_claimed_job,
80         _schedule_next_drain_if_waiting,
81     )
82
83     # Legacy thin wrapper for any remaining direct calls during transition.
84     # New code should import load_app_config / AppConfig from backend.config directly.
85     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
86         cfg = load_app_config(config_path)
87         return cfg.model_dump(mode="json")

```

```

88
89 # --- API Models ---
90 class ProcessRequest(BaseModel):
91     video_path: str
92     player_pool: Optional[List[str]] = None
93     max_frames: Optional[int] = None
94     segmenter: Optional[str] = None
95
96 class QueryRequest(BaseModel):
97     video_id: str
98     server: Optional[str] = None
99     receiver: Optional[str] = None
100     duration_min: Optional[float] = None
101     event_type: Optional[str] = None
102
103 class CompileRequest(BaseModel):
104     video_id: str
105     segment_ids: List[int]
106     output_filename: str
107
108 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
109     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
110
111     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
112     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
113     and
114     keep the prior good results intact ? a failed/empty re-run never destroys prior
115     data.
116     """
117     if segments:
118         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
119     else:
120         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
121
122 # --- API Endpoints ---
123 @app.get("/api/config")
124 def get_config():
125     return global_config
126
127 @app.get("/api/health")
128 def health():
129     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
130     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
131     confirm the engine is reachable and which detector is wired. Never raises."""
132     sport = getattr(global_config, "sport", None)
133     indexing = getattr(global_config, "indexing", None)
134     return {
135         "status": "ok",
136         "sport": sport,
137         "default_segmenter": getattr(indexing, "default_segmenter", None),
138     }
139
140 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141     """The full hash ? validate ? segment ? report pipeline for one video.
142
143     This is the former synchronous /api/process body, unchanged in behavior, now run on
144     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145     the sync endpoint used to return (UI compatibility); the per-video observability
146     report is still emitted in `finally:` and grafted as response["reports"].
147
148     `progress` is an optional callable(stage: str) used to surface coarse job progress.
149     Raises on unexpected internal errors ? the caller records those as a job failure
150     (after this function has already restored the pre-run segments, see below).
151     """

```

```

151     notify = progress or (lambda stage: None)
152
153     notify("hashing")
154     video_id = compute_video_id(req.video_path)
155     was_reingest = db_instance.get_video(video_id) is not None
156
157     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
158     notify("validating")
159     logger.info(f"Running validations for {req.video_path}...")
160     val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
161     failures = [r for r in val_results if not r.passed]
162
163     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
164     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
165     seg_name = req.segmenter or default_seg
166     segmenter_actual = None
167     segmentation_ran = False
168     response: Optional[Dict[str, Any]] = None
169     try:
170         if failures:
171             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
172             # status; it no longer destroys the video's prior good segments.
173             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
174             response = {
175                 "video_id": video_id,
176                 "status": "failed",
177                 "message": "Video failed validation checks.",
178                 "results": [r.model_dump() for r in val_results],
179             }
180             return response
181
182     # 2. Run segmenter & indexer
183     logger.info("[API] Video passed checks. Segmenting and indexing...")
184     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
185
186     segmenter_cls = segmenter_registry.get(seg_name)
187     if not segmenter_cls:
188         # Submit-time validation already 400s unknown names; this is the defensive
189         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
190         available = list(segmenter_registry.list_available().keys())
191         response = {
192             "video_id": video_id,
193             "status": "failed",
194             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
195             "results": [r.model_dump() for r in val_results],
196             "play_segments": [],
197         }
198         return response
199
200     logger.info(f"[API] Using segmenter: {seg_name}")
201     notify(f"segmenting ({seg_name})")
202     segmenter_actual = seg_name
203     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
204     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
205     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
206     segmenter = segmenter_cls(db_instance, global_config)

```

```

207         try:
208             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
209         except Exception:
210             # A raising segmenter must not leave half-written rows: restore the pre-run
211             # state (same destroy-after-confirm contract as the Failure branch), then
212             let
213                 # the job worker record the failure.
214                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
215                 raise
216             segmentation_ran = True
217
218         if isinstance(result, Failure):
219             logger.error(f"[API] Segmenter failed: {result.message}")
220             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
221             response = {
222                 "video_id": video_id,
223                 "status": "failed",
224                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
225                 "results": [r.model_dump() for r in val_results],
226                 "play_segments": [],
227             }
228             return response
229
230         segments = result.value if hasattr(result, "value") else result
231         notify("finalizing")
232         _finalize_reingest(video_id, segments, play_wm, cand_wm)
233
234         response = {
235             "video_id": video_id,
236             "status": "success",
237             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
238             "results": [r.model_dump() for r in val_results],
239             "play_segments": segments,
240         }
241         return response
242
243     finally:
244         # Always emit the per-video observability report (next to the video, or to
245         # report.output_dir). Never raises. Surface the paths in the response.
246         paths = emit_indexer_report(
247             db=db_instance, video_id=video_id, video_path=req.video_path,
248             config=global_config,
249             val_results=val_results, val_context=val_context,
250             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
251             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
252         )
253         if paths and isinstance(response, dict):
254             response["reports"] = paths
255
256
257 @app.post("/api/process", status_code=202)
258 def process_video(req: ProcessRequest):
259     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
260
261     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
262     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
263     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
264     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
265     """
266     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
267     try:
268         validate_input_path(req.video_path, allowed_root=allowed_root)
269     except ValueError as e:

```

```

267         raise HTTPException(status_code=400, detail=str(e)) from e
268     if not os.path.exists(req.video_path):
269         raise HTTPException(status_code=404, detail=f"Video file not found at
270 '{req.video_path}'")
271     if req.segmenter and not segmenter_registry.get(req.segmenter):
272         available = list(segmenter_registry.list_available().keys())
273         raise HTTPException(
274             status_code=400,
275             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
276         )
277     job_id = uuid.uuid4().hex
278     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
279                                 req.player_pool, req.max_frames):
280         raise HTTPException(status_code=500, detail="Failed to persist job record.")
281     _get_executor().submit(_drain_due_jobs)
282     return JSONResponse(
283         status_code=202,
284         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
285     )
286
287
288 @app.get("/api/jobs/{job_id}")
289 def get_job(job_id: str):
290     """Job state: {status, progress, result, reports}. `status` = execution state
291     (queued|running|done|failed); the pipeline verdict is result["status"]."""
292     job = db_instance.get_job(job_id)
293     if not job:
294         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
295     result = job.get("result")
296     return {
297         "job_id": job["id"],
298         "status": job["status"],
299         "progress": job.get("progress"),
300         "video_id": job.get("video_id"),
301         "error": job.get("error"),
302         "result": result,
303         "reports": (result or {}).get("reports"),
304         "created_at": job.get("created_at"),
305         "started_at": job.get("started_at"),
306         "finished_at": job.get("finished_at"),
307     }
308
309 # --- Chunked upload (B2, PR-2) -----
310 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
311 in
312 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
313 via
314 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
315 # sequential-part logic, and abort cleanup ? is unchanged.
316 from backend.api import uploads as uploads_api
317
318 app.include_router(uploads_api.router)
319
320
321 # --- Highlight download (B3, PR-2) -----
322
323 @app.get("/api/highlights/{video_id}/{filename}")
324 def download_highlight(video_id: str, filename: str):
325     """Stream a compiled highlight reel ? `filename` is the bare name given to
326 /api/compile
327     (which only writes into output_dir; the browser previously got back an unreachable
328     server-local path)."""
329     try:

```

```

327         name = safe_output_filename(filename)
328     except ValueError as e:
329         raise HTTPException(status_code=400, detail=str(e)) from e
330     if not db_instance.get_video(video_id):
331         raise HTTPException(status_code=404, detail="Indexed video record not found.")
332     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
333     or "output"
334     path = os.path.join(output_dir, name)
335     try:
336         path = resolve_under_root(path, output_dir)
337     except ValueError as e:
338         raise HTTPException(status_code=400, detail=str(e)) from e
339     if not os.path.isfile(path):
340         raise HTTPException(status_code=404,
341                             detail=f"No compiled file named '{name}'. Compile first via
342 /api/compile.")
343     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
344     return FileResponse(path, media_type=media, filename=name)
345
346 @app.post("/api/query")
347 def query_play_segments(req: QueryRequest):
348     filters = {
349         "server": req.server,
350         "receiver": req.receiver,
351         "duration_min": req.duration_min,
352         "event_type": req.event_type
353     }
354     segments = db_instance.query_play_segments(req.video_id, filters)
355     return {"play_segments": segments}
356
357 @app.post("/api/compile")
358 def compile_highlights(req: CompileRequest):
359     video = db_instance.get_video(req.video_id)
360     if not video:
361         raise HTTPException(status_code=404, detail="Indexed video record not found.")
362
363     # Retrieve matching play segments from db
364     all_segments = db_instance.query_play_segments(req.video_id, {})
365     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
366
367     if not matched_segments:
368         raise HTTPException(status_code=400, detail="No matching play segments found to
369 stitch.")
370
371     # Reject path traversal / absolute output names (os.path.join would otherwise let an
372 # absolute or '..'-laden output_filename write anywhere on disk).
373     try:
374         out_name = safe_output_filename(req.output_filename)
375     except ValueError as e:
376         raise HTTPException(status_code=400, detail=str(e)) from e
377
378     # The configured shot-count floor (stitching.min_shot_count) applies on the API
379 # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
380 shot_count
381 # metadata are exempt: the floor filters known counts only, so explicitly
382 # requested manual/legacy segments can't silently vanish under the default floor.
383 stitching = getattr(global_config, "stitching", None) or StitchingConfig()
384 kept = []
385 for s in matched_segments:
386     meta = s.get("metadata") or {}
387     sc = meta.get("shot_count") if isinstance(meta, dict) else None
388     try:
389         if sc is not None and int(sc) < stitching.min_shot_count:
390             continue

```



```

388         except (TypeError, ValueError):
389             pass # unparseable count -> treat as unknown, keep
390         kept.append(s)
391     if len(kept) < len(matched_segments):
392         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
393                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
394     if not kept:
395         raise HTTPException(status_code=400,
396                             detail=f"All {len(matched_segments)} matching segments fall
below "
397                                 f"stitching.min_shot_count={stitching.min_shot_count}.")
398
399     # Extract start/end time pairs
400     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
401
402     # Run compiler with the configured stitching paddings + optional branding watermark
403     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
404     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
405                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
406     try:
407         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
408         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
409         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
$4.3).
410         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
411         return {"status": "success", "filepath": out_path, "download_url": download_url}
412     except Exception as e:
413         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
414
415     # --- CLI Debug Utility & Orchestration ---
416     def run_cli_diagnose_clip(video_path: str, timestamp: float):
417         """CLI debugging utility to examine segment properties around a timestamp."""
418         print("-" * 60)
419         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
420         print("-" * 60)
421
422         cfg = load_config() # legacy wrapper still works, returns dict for now
423
424         # 1. Validation check diagnostics
425         print("[DIAGNOSTIC] Running validation framework...")
426         results = registry.run_all(video_path, cfg)
427         for r in results:
428             status_symbol = "[PASS]" if r.passed else "[FAIL]"
429             print(f" {status_symbol} {r.validator_name}: {r.message}")
430             if r.details:
431                 print(f"         Details: {r.details}")
432
433         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
434         print("-" * 60)
435         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
436         import cv2
437         cap = cv2.VideoCapture(video_path)
438         if not cap.isOpened():
439             print("[FAIL] OpenCV could not open video.")
440             return
441
442         fps = cap.get(cv2.CAP_PROP_FPS)
443         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

```

```

444
445     start_frame = max(0, int((timestamp - 3.0) * fps))
446     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
447
448     # Measure frame-by-frame changes in sample region
449     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
450     prev_gray = None
451     motions = []
452
453     for _ in range(start_frame, end_frame):
454         ret, frame = cap.read()
455         if not ret:
456             break
457         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
458         gray = cv2.GaussianBlur(gray, (21, 21), 0)
459
460         if prev_gray is not None:
461             diff = cv2.absdiff(prev_gray, gray)
462             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
463             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
464             motions.append(motion_ratio)
465             prev_gray = gray
466
467     cap.release()
468
469     if motions:
470         avg_motion = sum(motions) / len(motions)
471         # Support both legacy dict (from wrapper) and future direct model
472         if hasattr(cfg, "indexing"):
473             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
474         else:
475             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
476         print(f" Sample Frame Count: {len(motions)}")
477         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
478         if avg_motion > threshold:
479             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
480         else:
481             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
482     else:
483         print(" [ERROR] No visual frames were extracted in the sample range.")
484
485     print("=" * 60)
486
487     def main():
488         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
489         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
490         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
491         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
492         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
493         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
494         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
495         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
496

```

```

497     available_segmenters = list(segmenter_registry.list_available().keys())
498     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
499     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
500     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
501     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
502
503     args = parser.parse_args()
504
505     if args.setup:
506         # Invoke the diagnostics/bootstrap script (renamed from the old root
507         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
508         # build system). Resolve its path from this file so `indexer --setup`
509         # works regardless of the caller's cwd.
510         import subprocess
511         doctor_path = os.path.join(
512             os.path.dirname(os.path.abspath(__file__)),
513             "scripts", "doctor.py",
514         )
515         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
516         subprocess.run([sys.executable, doctor_path])
517         return
518
519     if args.trim_start is not None and args.trim_end is not None:
520         if not args.video_path:
521             print("[ERROR] Please provide --video-path to extract a segment.")
522             return
523
524         # Determine output path
525         out_filename = args.trim_output or "trimmed_segment.mp4"
526         if os.path.isabs(out_filename):
527             out_path = out_filename
528             out_dir = os.path.dirname(out_path)
529             out_name = os.path.basename(out_path)
530         else:
531             out_dir = args.output_dir
532             out_path = os.path.join(out_dir, out_filename)
533             out_name = out_filename
534
535         os.makedirs(out_dir, exist_ok=True)
536         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
537
538         # global_config isn't initialized this early; load the typed config so the trim
539         # clip carries the same configured stitching paddings + watermark as a real
540         # highlight (watermark default-on).
541         stitching = load_app_config().stitching
542         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
543                                 end_padding=stitching.end_padding,
544                                 watermark=stitching.watermark)
545         try:
546             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
547             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
548         except Exception as e:
549             print(f"[ERROR] Failed to compile trimmed segment: {e}")
550         return
551
552     if args.debug_clip is not None:
553         if not args.video_path:
554             print("[ERROR] Please provide --video-path to debug a specific clip.")
555             return

```

```

555         run_cli_diagnose_clip(args.video_path, args.debug_clip)
556         return
557
558     # Initialize Global Config and DB
559     global global_config, db_instance
560     global_config = load_app_config() # returns typed AppConfig
561
562     # The CLI --output-dir overrides the configured output directory. It now lives
563     # on the typed model (OutputConfig.output_dir), so every consumer ? including
564     # /api/compile ? reads the same value instead of a write-only runtime hack.
565     global_config.output.output_dir = args.output_dir
566     os.makedirs(global_config.output.output_dir, exist_ok=True)
567
568     db_path = os.path.join(global_config.output.output_dir,
569 global_config.output.db_name)
570     db_instance = Database(db_path)
571
572     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
573     # the executor is in-process. Mark them failed so pollers see a terminal state.
574     swept = db_instance.fail_stale_jobs()
575     if swept:
576         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
577 failed.")
578
579     # CLI processing mode (no web UI needed)
580     if args.video_path and not args.server:
581         print(f"[CLI] Processing video file: {args.video_path}")
582         if not os.path.exists(args.video_path):
583             print(f"[FAIL] Video file not found: {args.video_path}")
584             return
585
586         video_id = compute_video_id(args.video_path)
587         was_reingest = db_instance.get_video(video_id) is not None
588
589         # Validate (with-context variant surfaces ffprobe_meta for the report)
590         print("[CLI] Validating...")
591         val_results, val_context = registry.run_all_with_context(args.video_path,
592 global_config)
593         failures = [r for r in val_results if not r.passed]
594
595         # Save video status
596         status = "failed" if failures else "processed"
597         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
598 in val_results])
599
600         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
601 "default_segmenter", "motion")
602         segmenter_actual = None
603         segmentation_ran = False
604         try:
605             print("\n" + "="*40)
606             print("VALIDATION SUMMARY")
607             print("="*40)
608             for r in val_results:
609                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
610                 print(f"{status_symbol} {r.validator_name}: {r.message}")
611             print("="*40 + "\n")
612
613             if failures:
614                 print("[FAIL] Video failed checks. Indexing halted.")
615                 return
616
617         # Index
618         segmenter_cls = segmenter_registry.get(seg_name)
619         if not segmenter_cls:

```

```

615         print(f"[FAIL] Segmenter '{seg_name}' not found.")
616         return
617
618         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
619         segmenter_actual = seg_name
620         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
621         segmenter = segmenter_cls(db_instance, global_config)
622         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
623         segmentation_ran = True
624
625         if isinstance(result, Failure):
626             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
627             print("[FAIL] Indexing halted for video.")
628             segments = []
629         else:
630             segments = result.value if hasattr(result, "value") else result
631             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
632             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
633             _finalize_reingest(video_id, segments, play_wm, cand_wm)
634         finally:
635             # Always emit the per-video observability report. Never raises.
636             paths = emit_indexer_report(
637                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
638                 val_results=val_results, val_context=val_context,
639                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
640                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
641             )
642             if paths:
643                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
644                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
645                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
646             return
647
648         # Start Local Server Mode
649         if args.server or not args.video_path:
650             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
651             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
652             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
653             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
654             _get_executor().submit(_drain_due_jobs)
655             uvicorn.run(app, host="127.0.0.1", port=args.port)
656
657     if __name__ == "__main__":
658         main()
659

```

10. assistant (2026-06-21T07:40:09.629Z)

I have the SPA client and the main engine API. Note the key finding already: `/api/compile` at line 411 returns `download\_url` (the "M1" addition), but the SPA's `CompileResponse` type (types.ts:33-37) only knows `status` + `filepath` ? it hand-builds the URL with `highlightUrl()`. Now let me read the worker, storage backends, uploads, config, and the design docs in parallel.

11. user (2026-06-21T07:40:10.582Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2

```

```

3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9      are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16     from __future__ import annotations
17
18     import argparse
19     import csv
20     import json
21     import logging
22     import os
23     import sys
24     import tempfile
25     from typing import Any, List
26
27     from backend import storage
28     from backend.config import load_config
29     from backend.infrastructure.database import Database
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.validators.base import registry as validator_registry
32     from backend.reporting import emit_indexer_report
33     from backend.results import Failure
34     from backend.utils.hashing import compute_video_id
35
36     logger = logging.getLogger("backend.worker")
37
38
39     def _coerce(v: str) -> Any:
40         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
41         low = v.lower()
42         if low in ("true", "false"):
43             return low == "true"
44         for cast in (int, float):
45             try:
46                 return cast(v)
47             except ValueError:
48                 pass
49         return v
50
51
52     def _apply_overrides(config: Any, sets: List[str]) -> None:
53         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
54         for kv in sets or []:
55             key, sep, val = kv.partition("=")
56             if not sep:
57                 raise ValueError(f"--set expects k=v, got {kv!r}")
58             parts = key.split(".")
59             obj = config
60             for p in parts[:-1]:
61                 obj = getattr(obj, p)

```

```

62         setattr(obj, parts[-1], _coerce(val))
63
64
65     def _write_intervals_csv(segments: List[dict], path: str) -> None:
66         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
67         artifact (same schema as the rally-annotator labels)."""
68         with open(path, "w", newline="") as f:
69             w = csv.writer(f)
70             w.writerow(["start", "end", "shot_count", "ending_reason"])
71             for s in segments:
72                 w.writerow([
73                     f'{float(s.get("start_time", 0.0)):.3f}',
74                     f'{float(s.get("end_time", 0.0)):.3f}',
75                     s.get("shot_count", ""),
76                     s.get("ending_reason", s.get("shot_count_confidence", "")),
77                 ])
78
79
80     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
81         with open(path, "w") as f:
82             json.dump({
83                 "video_id": video_id,
84                 "status": status,
85                 "segment_count": len(segments),
86                 "segments": [{"start": float(s.get("start_time", 0.0)),
87                             "end": float(s.get("end_time", 0.0))} for s in segments],
88             }, f, indent=2)
89
90
91     def cmd_infer(args: argparse.Namespace) -> int:
92         config = load_config(args.config) if args.config else load_config()
93         _apply_overrides(config, args.set)
94         storage_cfg = config.storage.model_dump()
95
96         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
97         os.makedirs(scratch, exist_ok=True)
98         config.output.output_dir = scratch
99
100        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
101        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
102        print(f"[worker] pulled {args.video_uri} -> {local_video}")
103
104        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
105        video_id = compute_video_id(local_video)
106
107        # 3. VALIDATE (same registry as the main CLI).
108        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
109        failures = [r for r in val_results if not r.passed]
110        db = Database(os.path.join(scratch, config.output.db_name))
111        db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
112
113        segments: List[dict] = []
114        segmenter_actual = None
115        segmentation_ran = False
116        status = "failed"
117        try:
118            if failures:
119                print("[worker] validation FAILED: ")
120

```

```

121         + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
122     return 2
123     seg_name = args.segmenter or config.indexing.default_segmenter
124     segmenter_cls = segmenter_registry.get(seg_name)
125     if not segmenter_cls:
126         print(f"[worker] unknown segmenter: {seg_name!r} "
127               f"(have {list(segmenter_registry.list_available())})")
128     return 2
129     segmenter_actual = seg_name
130     result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
131     segmentation_ran = True
132     if isinstance(result, Failure):
133         print(f"[worker] segmenter FAILED: {result.message}")
134         return 3
135     segments = result.value if hasattr(result, "value") else result
136     status = "success"
137     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
138     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
139     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
140     if not segments:
141         detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
142                          or getattr(config.indexing, "detector_impl", None) or
"auto")
143         logger.warning(
144             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
145             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
146             "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
147             "the WASB env produced no detections, or the input resolution differs
from the tuned "
148             "proxy resolution. Re-run with -v for the native command + frame
counts.",
149             video_id, segmenter_actual, detector_impl,
150         )
151     finally:
152         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
153         emit_indexer_report(
154             db=db, video_id=video_id, video_path=local_video, config=config,
155             val_results=val_results, val_context=val_context,
156             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
157             was_reingest=False, segmentation_ran=segmentation_ran,
158         )
159
160     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
161     out_local = os.path.join(scratch, "outputs", video_id)
162     os.makedirs(out_local, exist_ok=True)
163     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
164     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
165
166     out_prefix = args.out_uri.rstrip("/")
167     pushed = []
168     for fn in sorted(os.listdir(out_local)):
169         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
170         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
171         pushed.append(uri)
172
173     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}

```



```

artifact(s):")
174         for u in pushed:
175             print("    ", u)
176         return 0
177
178
179     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
180     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
181     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
182
183
184     def _probe_video_fps_width(path: str):
185         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
186
187         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
188         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
189         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
190         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
191         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
192         rather than guesses.
193         """
194         import json
195         import subprocess
196         try:
197             out = subprocess.check_output(
198                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
199                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
200                 stderr=subprocess.DEVNULL).decode()
201             st = (json.loads(out).get("streams") or [{}])[0]
202             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
203             fps = float(num) / float(den or "1")
204             width = float(st.get("width") or 0)
205             if fps > 0 and width > 0:
206                 return fps, width
207         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
208             pass
209         return None
210
211
212     def _resolve_train_detector(args: argparse.Namespace, config: Any):
213         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
214         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
215         stub=CPU mock). Returns the runner, or prints an error + returns None.
216
217         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
218         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
219         no shuttle detector and is rejected.
220         """
221         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
222
223         seg_cls = segmenter_registry.get(args.segmenter)
224         if not seg_cls:
225             print(f"[worker] unknown segmenter: {args.segmenter!r} "
226                   f"(have {list(segmenter_registry.list_available())})")

```

```

227         return None
228     seg = seg_cls(None, config)
229     if not isinstance(seg, TrajectoryHybridSegmenter):
230         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
231               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / "
232               f"fusion_hybrid).")
233         return None
234     try:
235         runner, _ = seg._resolve_runner(config.get("indexing", {}))
236     except NotImplementedError as e:
237         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
238         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
239         traceback.
240         print(f"[worker] detector not available: {e}")
241         return None
242     ok, msg = runner.healthcheck()
243     if not ok:
244         print(f"[worker] detector environment not ready: {msg}")
245         return None
246     print(f"[worker] detector ready ({args.segmenter}): {msg}")
247     return runner
248
249 def cmd_train(args: argparse.Namespace) -> int:
250     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
251     shell out
252     to training.gen0.harness (backend must NOT import training) -> push the artifact
253     bundle.
254     Two trajectory sources (the train pipeline + harness are identical either way):
255     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
256     synthetic
257     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
258     CI.
259     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
260     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
261     This
262     is the M3.5 "first real training" path; only the trajectory source flips.
263     """
264     import subprocess
265
266     config = load_config(args.config) if args.config else load_config()
267     _apply_overrides(config, args.set)
268     storage_cfg = config.storage.model_dump()
269
270     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
271     labels_dir = os.path.join(scratch, "labels")
272     traj_dir = os.path.join(scratch, "trajectories")
273     videos_dir = os.path.join(scratch, "videos")
274     os.makedirs(labels_dir, exist_ok=True)
275     os.makedirs(traj_dir, exist_ok=True)
276
277     corpus = args.corpus_uri.rstrip("/")
278     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
279 config=storage_cfg)
280                          if u.lower().endswith(".csv"))
281     if len(label_uris) < 2:
282         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
283 {len(label_uris)} "
284               f"under {corpus}/labels/")
285     return 2
286
287 # Real-detector source: resolve the runner once + index the bucket's videos/ by
288 stem.
289 runner = None

```

```

282     video_by_stem: dict = {}
283     if args.trajectory_source == "detector":
284         os.makedirs(videos_dir, exist_ok=True)
285         runner = _resolve_train_detector(args, config)
286         if runner is None:
287             return 2
288         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
289             vname = storage.parse_uri(vuri).name
290             if not vname.lower().endswith(_VIDEO_EXTS):
291                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
292                 video_by_stem[os.path.splitext(vname)[0]] = vuri
293
294     print(f"[worker] trajectory source: {args.trajectory_source}")
295     specs: List[dict] = []
296     for uri in label_uris:
297         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
298         name = fname.replace(".rallies.csv", "").replace(".csv", "")
299         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
300         if args.trajectory_source == "detector":
301             vuri = video_by_stem.get(name)
302             if not vuri:
303                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
304                 continue
305             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
306                                     config=storage_cfg)
307             video_id = compute_video_id(local_video)
308             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
309             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
310             meta = _probe_video_fps_width(local_video)
311             if meta is None:
312                 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
313                 continue
314             fps, frame_width = meta
315             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
316             if not traj:
317                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
318                 continue
319             specs.append({"name": name, "trajectory": traj, "labels": local_label,
320                          "fps": fps, "frame_width": frame_width})
321         else:
322             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
323             traj = os.path.join(traj_dir, f"{name}_traj.csv")
324             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
325             specs.append({"name": name, "trajectory": traj, "labels": local_label,
326                          "fps": args.fps, "frame_width": args.frame_width})
327
328     if len(specs) < 2:
329         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
330         return 2
331     manifest_path = os.path.join(scratch, "manifest.json")
332     with open(manifest_path, "w", encoding="utf-8") as f:
333         json.dump(specs, f, indent=2)
334     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock

```

```

stub"
335     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
336
337     out_dir = os.path.join(scratch, "artifacts")
338     cmd = [sys.executable, "-m", "training.gen0.harness",
339           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
340     if args.floor:
341         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
342 config=storage_cfg)
343                        if "://" in args.floor else args.floor)
344         cmd += ["--floor", floor_local]
345     print("[worker] training:", " ".join(cmd))
346     rc = subprocess.run(cmd).returncode
347     if rc != 0:
348         print(f"[worker] gen0 harness failed (rc={rc})")
349         return rc
350
351     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
352     bundle = os.path.join(out_dir, args.version)
353     out_prefix = args.out_uri.rstrip("/")
354     pushed = []
355     for fn in sorted(os.listdir(bundle)):
356         uri = f"{out_prefix}/weights/{args.version}/{fn}"
357         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
358         pushed.append(uri)
359     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
360     for u in pushed:
361         print("    ", u)
362     return 0
363
364 def build_parser() -> argparse.ArgumentParser:
365     p = argparse.ArgumentParser(prog="python -m backend.worker",
366                                description="Storage-aware worker: pull ? run pipeline ?
367 push.")
368     p.add_argument("-v", "--verbose", action="store_true",
369                   help="DEBUG logging (the native detector command, frame counts, per-
370 stage detail)")
371     sub = p.add_subparsers(dest="cmd", required=True)
372
373     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
374     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
375 gcs://")
376     pi.add_argument("--out-uri", required=True, help="output prefix
377 (outputs/<video_id>/? is appended)")
378     pi.add_argument("--segmenter", default=None, help="default:
379 indexing.default_segmenter")
380     pi.add_argument("--config", default=None, help="config.json path (default:
381 ./config.json)")
382     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
383 dir)")
384     pi.add_argument("--max-frames", type=int, default=None)
385     pi.add_argument("--set", action="append", default=[], metavar="k=v",
386                   help="config override, e.g. --set indexing.skip_ai_handoff=true")
387     pi.set_defaults(func=cmd_infer)
388
389     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
390 trajectories -> harness)")
391     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
392 videos/ for "
393                    "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
394     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
395 is appended)")
396     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
397     pt.add_argument("--trajectory-source", choices=["synth", "detector"],

```

```

default="synth",
388             help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
389             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
390             "(needs videos/ in the corpus; honours detector_impl).")
391     pt.add_argument("--segmenter", default="wasb_hybrid",
392             help="trajectory hybrid whose detector is used for --trajectory-
source detector")
393     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
394     pt.add_argument("--config", default=None)
395     pt.add_argument("--scratch", default=None)
396     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
397     pt.add_argument("--frame-width", type=float, default=1280.0,
398             help="synth trajectory frame width (detector probes the video)")
399     pt.add_argument("--set", action="append", default=[], metavar="k=v")
400     pt.set_defaults(func=cmd_train)
401     return p
402
403
404     def _setup_logging(verbose: bool) -> None:
405         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
406         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
407         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
408         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
409         entry), not at import, so importing the module never reconfigures a caller's
logging."""
410         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
411         # configured logging at import), which would leave the level at WARNING and re-
suppress the
412         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
413         logging.basicConfig(
414             level=logging.DEBUG if verbose else logging.INFO,
415             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
416             datefmt="%H:%M:%S",
417             force=True,
418         )
419
420
421     def main(argv=None) -> int:
422         args = build_parser().parse_args(argv)
423         _setup_logging(getattr(args, "verbose", False))
424         return args.func(args)
425
426
427     if __name__ == "__main__":
428         sys.exit(main())
429

```

12. user (2026-06-21T07:40:11.549Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart

```

```

7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """
15    import os
16    import threading
17    import uuid
18    from typing import Any, Dict, Optional
19
20    from fastapi import APIRouter, HTTPException, Request
21    from pydantic import BaseModel
22
23    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24    from backend.utils.paths import safe_output_filename
25
26    router = APIRouter()
27
28    _uploads: Dict[str, Dict[str, Any]] = {}
29    _uploads_lock = threading.Lock()
30
31
32    class UploadInitRequest(BaseModel):
33        filename: str
34        total_size_bytes: Optional[int] = None
35
36
37    class UploadCompleteRequest(BaseModel):
38        total_parts: int
39
40
41    def _upload_cfg():
42        # Resolved lazily (import inside the function) to read the LIVE startup config that
43        # main() sets on the module global, and to avoid a circular import at module load
44        # (main.py imports this router; this module must not import main at import time).
45        import backend.main as _main
46        sec = getattr(_main.global_config, "security", None)
47        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51        None) or ["mp4", "mov"])]
52        return upload_dir, max_bytes, part_bytes, exts
53
54    def _abort_upload(upload_id: str) -> None:
55        with _uploads_lock:
56            st = _uploads.pop(upload_id, None)
57            if st:
58                try:
59                    os.remove(st["tmp_path"])
60                except OSError:
61                    pass
62
63
64    @router.post("/api/upload/init")
65    def upload_init(req: UploadInitRequest):
66        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68        try:
69            name = safe_output_filename(req.filename)
70            except ValueError as e:

```

```

71         raise HTTPException(status_code=400, detail=str(e)) from e
72     ext = os.path.splitext(name)[1].lower().lstrip(".")
73     if ext not in exts:
74         raise HTTPException(status_code=400,
75                             detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
76     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77         raise HTTPException(status_code=413,
78                             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79     os.makedirs(upload_dir, exist_ok=True)
80     upload_id = uuid.uuid4().hex
81     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82     open(tmp_path, "wb").close()
83     with _uploads_lock:
84         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86           "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89     @router.put("/api/upload/{upload_id}/part/{index}")
90     async def upload_part(upload_id: str, index: int, request: Request):
91         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92         _, max_bytes, part_bytes, _ = _upload_cfg()
93         with _uploads_lock:
94             st = _uploads.get(upload_id)
95             if st is None:
96                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97             if index != st["parts"]:
98                 raise HTTPException(status_code=409,
99                                     detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101             received = 0
102             overflow = None
103             with open(st["tmp_path"], "ab") as fh:
104                 async for chunk in request.stream():
105                     received += len(chunk)
106                     if received > part_bytes:
107                         overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                         break
109                     if st["bytes"] + received > max_bytes:
110                         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                         break
112                     fh.write(chunk)
113             if overflow:
114                 _abort_upload(upload_id)
115                 raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117             with _uploads_lock:
118                 st["parts"] += 1
119                 st["bytes"] += received
120             return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()

```

```

127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129             if st is None:
130                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131             if st["parts"] == 0 or st["parts"] != req.total_parts:
132                 raise HTTPException(status_code=409,
133                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134             final_path = os.path.join(upload_dir, st["filename"])
135             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136                 stem, ext = os.path.splitext(st["filename"])
137                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138             os.replace(st["tmp_path"], final_path)
139             with _uploads_lock:
140                 _uploads.pop(upload_id, None)
141             video_id = compute_video_id(final_path)
142             return {"path": os.path.abspath(final_path), "video_id": video_id,
143                     "size_bytes": os.path.getsize(final_path),
144                     "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152             if not known:
153                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154             _abort_upload(upload_id)
155             return {"status": "aborted", "upload_id": upload_id}
156

```

13. user (2026-06-21T07:40:13.082Z)

```

=== storage dir ===
total 44
drwxr-xr-x 1 avidu 197609    0 Jun 20 15:47 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 06:12 ..
-rw-r--r-- 1 avidu 197609 3454 Jun 20 15:34 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 13:02 __pycache__
-rw-r--r-- 1 avidu 197609 2231 Jun 20 15:34 base.py
-rw-r--r-- 1 avidu 197609 2661 Jun 20 13:48 gcs.py
-rw-r--r-- 1 avidu 197609 6910 Jun 20 15:47 gdrive.py
-rw-r--r-- 1 avidu 197609 2756 Jun 20 13:48 local.py
-rw-r--r-- 1 avidu 197609 2542 Jun 20 15:37 ref.py
-rw-r--r-- 1 avidu 197609 3728 Jun 20 13:48 s3.py
=== segmenters dir ===
total 125
drwxr-xr-x 1 avidu 197609    0 Jun 20 14:39 .
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 ..
-rw-r--r-- 1 avidu 197609   757 Jun 14 01:12 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 13:02 __pycache__
-rw-r--r-- 1 avidu 197609   546 Jun 20 13:48 _keyhint.py
-rw-r--r-- 1 avidu 197609  2420 Jun 11 10:48 base.py
-rw-r--r-- 1 avidu 197609 11618 Jun 20 14:17 fusion_hybrid.py
-rw-r--r-- 1 avidu 197609 25418 Jun 20 13:48 gemini.py
-rw-r--r-- 1 avidu 197609 11019 Jun 19 13:03 motion.py
-rw-r--r-- 1 avidu 197609  1706 Jun 11 11:10 tracknet_hybrid.py
-rw-r--r-- 1 avidu 197609 20264 Jun 20 14:39 trajectory_hybrid.py
-rw-r--r-- 1 avidu 197609  2280 Jun 20 14:17 wasb_hybrid.py
-rw-r--r-- 1 avidu 197609 21263 Jun 20 13:48 yolo_hybrid.py
=== worker dir ===
total 33
drwxr-xr-x 1 avidu 197609    0 Jun 21 08:12 .

```



```
drwxr-xr-x 1 avidu 197609      0 Jun 21 06:12 ..
-rw-r--r-- 1 avidu 197609    363 Jun 20 13:48 __init__.py
-rw-r--r-- 1 avidu 197609 21752 Jun 21 08:12 __main__.py
drwxr-xr-x 1 avidu 197609      0 Jun 21 13:02 __pycache__
```

14. assistant (2026-06-21T07:40:15.917Z)

The storage backends exist: `gcs.py`, `gdrive.py`, `local.py`, `s3.py`, plus `base.py`, `\_\_init\_\_.py`, `ref.py`. Let me read the storage backends and the config to confirm GCS support and the detector wiring.

15. user (2026-06-21T07:40:17.151Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13     from __future__ import annotations
14
15     import os
16     from typing import Any, Dict, Iterator, Optional
17
18     from backend.storage.base import StorageBackend
19     from backend.storage.ref import StorageRef, StorageStat, parse_uri
20
21     __all__ = [
22         "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
23         "get_storage", "fetch", "put", "list_uris",
24     ]
25
26
27     def _backend_class(name: str):
28         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
29         if name == "local":
30             from backend.storage.local import LocalStorage
31             return LocalStorage
32         if name == "s3":
33             from backend.storage.s3 import S3Storage
34             return S3Storage
35         if name == "gcs":
36             from backend.storage.gcs import GCSStorage
37             return GCSStorage
38         if name == "gdrive":
39             from backend.storage.gdrive import GDriveStorage
40             return GDriveStorage
41         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
42
43
44     def get_storage(backend: Optional[str] = None, *,
45                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
46         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
```

```

47         the implementation; its per-backend settings come from ``config[backend]`` merged
with
48         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
49         cfg = config or {}
50         name = backend or cfg.get("backend", "local")
51         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
52         settings.update(overrides)
53         return _backend_class(name)(**settings)
54
55
56     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
57             config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
58         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
59         (no copy); for s3/gcs it downloads to ``dst``."""
60         ref = parse_uri(uri)
61         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
62
63
64     def put(local_path: "str | os.PathLike[str]", uri: str, *,
65           config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
66         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
67         ref = parse_uri(uri)
68         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
69         return uri
70
71
72     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
73         """Yield the URI of every object under ``prefix_uri``."""
74         ref = parse_uri(prefix_uri)
75         for r in get_storage(ref.backend, config=config).list(ref):
76             yield r.uri
77

```

16. user (2026-06-21T07:40:17.404Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta
11    from typing import Any, Iterator, Optional
12
13    from backend.storage.base import StorageBackend
14    from backend.storage.ref import StorageRef, StorageStat
15
16
17    class GCSStorage(StorageBackend):
18        def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                signed_url_ttl: int = 3600, **_ignored) -> None:
20            self._ttl = timedelta(seconds=signed_url_ttl)
21            if client is not None:

```

```

22         self._client = client
23         return
24         from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25         self._client = storage.Client(project=project)
26
27     def _blob(self, ref: StorageRef):
28         return self._client.bucket(ref.bucket).blob(ref.key)
29
30     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31         if dst is None:
32             raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33         import os
34         dst = str(dst)
35         if os.path.exists(dst) and not overwrite:
36             return dst
37         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38         self._blob(ref).download_to_filename(dst)
39         return dst
40
41     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43         return ref
44
45     def exists(self, ref) -> bool:
46         return bool(self._blob(ref).exists())
47
48     def stat(self, ref) -> StorageStat:
49         b = self._blob(ref)
50         b.reload()
51         return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
57
58     def delete(self, ref) -> None:
59         self._blob(ref).delete()
60
61     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62         return self._blob(ref).generate_signed_url(
63             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64

```

17. user (2026-06-21T07:40:17.877Z)

```

1     """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3     Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4     live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5     pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6     no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7     """
8     from __future__ import annotations
9
10    import os
11    from abc import ABC, abstractmethod
12    from typing import Iterator
13
14    from backend.storage.ref import StorageRef, StorageStat

```

```

15
16
17 class StorageBackend(ABC):
18     """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19     worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21     @abstractmethod
22     def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                       *, overwrite: bool = False) -> str:
24         """Make the object available as a local file; return its local path. For the
25         ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27     @abstractmethod
28     def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29            content_type: "str | None" = None) -> StorageRef:
30         """Upload/copy a local file to ``ref``; return ``ref``."""
31
32     @abstractmethod
33     def exists(self, ref: StorageRef) -> bool: ...
34
35     @abstractmethod
36     def stat(self, ref: StorageRef) -> StorageStat: ...
37
38     @abstractmethod
39     def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40         """Yield a ``StorageRef`` per object under ``prefix``."""
41
42     @abstractmethod
43     def delete(self, ref: StorageRef) -> None: ...
44
45     def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46         """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47         sign (``local`` cannot; Drive only emulates) ? callers must treat
48         ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49         raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50

```

18. user (2026-06-21T07:40:18.911Z)

```

1     """Local-filesystem storage backend ? the default, zero new dependencies.
2
3     Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4     file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5     the whole pipeline behaves exactly as before the storage layer existed.
6     """
7     from __future__ import annotations
8
9     import os
10    import shutil
11    from datetime import datetime
12    from typing import Iterator
13
14    from backend.storage.base import StorageBackend
15    from backend.storage.ref import StorageRef, StorageStat
16
17
18    class LocalStorage(StorageBackend):
19        def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20            # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21            self.base_dir = base_dir

```

```

22
23     def _path(self, ref: StorageRef) -> str:
24         p = ref.key
25         if self.base_dir and not os.path.isabs(p):
26             return os.path.join(self.base_dir, p)
27         return p
28
29     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30         # The object is already on the local FS ? return its path (identity). ``dst`` is
31         # honoured only if it names a *different* location (a copy is then made).
32         src = self._path(ref)
33         if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34             return src
35         dst = str(dst)
36         if os.path.exists(dst) and not overwrite:
37             return dst
38         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39         shutil.copyfile(src, dst)
40         return dst
41
42     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43         dst = self._path(ref)
44         if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46             shutil.copyfile(str(local_path), dst)
47         return ref
48
49     def exists(self, ref) -> bool:
50         return os.path.exists(self._path(ref))
51
52     def stat(self, ref) -> StorageStat:
53         st = os.stat(self._path(ref))
54         return StorageStat(size=st.st_size,
55 modified=datetime.fromtimestamp(st.st_mtime))
56
57     def list(self, prefix) -> Iterator[StorageRef]:
58         base = self._path(prefix)
59         if os.path.isdir(base):
60             for root, _dirs, files in os.walk(base):
61                 for fn in sorted(files):
62                     full = os.path.join(root, fn)
63                     yield StorageRef("local", "", full, f"local://{full}")
64             elif os.path.exists(base):
65                 yield StorageRef("local", "", base, f"local://{base}")
66
67     def delete(self, ref) -> None:
68         path = self._path(ref)
69         if os.path.exists(path):
70             os.remove(path)

```

19. assistant (2026-06-21T07:40:22.523Z)

GCS is a first-class, fully-implemented backend (download + upload + list + signed URLs). Now let me read the config (to see storage config and detector defaults), the segmenter registry/wasb/gemini detector wiring, and the two design docs.

20. user (2026-06-21T07:40:23.403Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5     <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)

```

```

6         local://<path>                -> local    (local:///abs or local://rel/p)
7         s3://<bucket>/<key>            -> s3        (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8         gcs://<bucket>/<key>          -> gcs        (gs:// accepted, normalized to gcs://)
9         gdrive://<path>                -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14     from __future__ import annotations
15
16     import re
17     from dataclasses import dataclass
18     from datetime import datetime
19     from typing import Optional
20
21     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22     _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25     @dataclass(frozen=True)
26     class StorageStat:
27         """Lightweight object metadata (uniform across backends)."""
28         size: int
29         modified: Optional[datetime] = None
30         etag: Optional[str] = None
31         content_type: Optional[str] = None
32
33
34     @dataclass(frozen=True)
35     class StorageRef:
36         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38         backend: str
39         bucket: str
40         key: str
41         uri: str
42
43         @property
44         def name(self) -> str:
45             return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48     def parse_uri(uri: str) -> StorageRef:
49         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52         have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53         """
54         s = str(uri)
55         m = _SCHEME_RE.match(s)
56         if not m:
57             return StorageRef("local", "", s, f"local://{s}")
58         scheme = m.group(1).lower()
59         rest = s[m.end():]
60         if scheme == "gs":
61             scheme = "gcs"
62         if scheme in ("local", "gdrive"):
63             # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".

```

```

64         return StorageRef(scheme, "", rest, s)
65     if scheme in _KNOWN:
66         bucket, _, key = rest.partition("/")
67         return StorageRef(scheme, bucket, key, s)
68     raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})}")
69

```

21. user (2026-06-21T07:40:24.354Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}

```

```

54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

22. user (2026-06-21T07:40:24.818Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
34             badminton "
35             "weights) for precise candidate rallies + AI provider for semantic
36             boundaries.")
37
38         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
39             Any]]:
40             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
41             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
42
43         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
44             Dict[str, Any]]:
45             # Lazy import: the native runner pulls no heavy deps, but keep parity with the
46             stub
47             # seam (imported only when detector_impl='native' is actually selected).
48             from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
49             NativeWasbConfig
50             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
51             return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,

```



```

46             "detector_impl": "native", "device": cfg.device}
47
48
49     segmenter_registry.register(WasbHybridSegmenter)
50

```

23. user (2026-06-21T07:40:25.151Z)

```

1     from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2     from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
8
9     __all__ = [
10         "segmenter_registry",
11         "BasePlaySegmenter",
12         "MotionPlaySegmenter",
13         "GeminiPlaySegmenter",
14         "HybridYoloSegmenter",
15         "TrackNetHybridSegmenter",
16         "WasbHybridSegmenter",
17         "FusionSegmenter",
18     ]
19

```

24. assistant (2026-06-21T07:40:28.126Z)

Now let me read the config (storage + indexing/detector defaults), the motion segmenter (the CPU default), and the two design docs. Let me also check how the detector_impl resolves (stub vs native vs auto/WSL) and what config.json looks like.

25. user (2026-06-21T07:40:29.938Z)

```

=== config dir ===
__init__.py
__pycache__
loader.py
models.py
presets.py
=== config.json (root) ===
{
  "sport": "badminton",
  "ai_provider": "gemini",
  "ai_model": "gemini-flash-latest",
  "validation": {
    "min_resolution_width": 1280,
    "min_resolution_height": 720,
    "min_fps": 29.9,
    "min_blur_threshold": 20.0,
    "max_scene_cuts_per_minute": 0.5,
    "relevance": {
      "court_green_lower": [
        35,
        40,
        40
      ],
      "court_green_upper": [
        85,
        255,
        255
      ],
    }
  }
}

```

```

        "court_pixels_min_ratio": 0.05
    }
},
"indexing": {
    "default_segmenter": "gemini",
    "use_gpu": true,
    "motion_threshold": 0.08,
    "min_segment_duration": 3.0,
    "dead_time_merge_gap": 2.0,
    "chunk_duration": 90,
    "chunk_overlap": 10,
    "detector_model": "fasterrcnn_resnet50_fpn",
    "detector_score_threshold": 0.5,
    "yolo_velocity_threshold": 0.15,
    "yolo_frame_skip": 3,
    "yolo_tracker": "bytetrack.yaml",
    "yolo_prefetch_workers": 2,
    "min_action_window_duration": 2.0,
    "log_yolo_candidates": true,
    "store_yolo_candidates": true,
    "ai_handoff_padding": 2.0,
    "ai_max_workers": 1,
    "tracknet": {
        "wsl_distro": "Ubuntu",
        "repo_dir": "~/models/TrackNetV4",
        "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
        "conda_env": "TrackNetV4",
        "weights_path": "",
        "queue_length": 5,
        "stage_in_wsl": true,
        "wsl_stage_dir": "~/clips",
        "timeout_sec": 1800,
        "velocity_threshold": 0.02
    },
    "wasb": {
        "wsl_distro": "Ubuntu",
        "repo_dir": "~/models/WASB-SBDT",
        "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
        "conda_env": "wasb",
        "weights_path": "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
        "sport": "badminton",
        "wsl_stage_dir": "~/clips",
        "timeout_sec": 1800,
        "velocity_threshold": 0.02
    }
},
"output": {
    "default_highlights_name": "highlights.mp4",
    "db_name": "sports_indexer.db"
},
"stitching": {
    "begin_padding": 0.5,
    "end_padding": 0.5,
    "use_cache": false,
    "min_shot_count": 1,
    "watermark": {
        "enabled": true,
        "text": "Generated by Khelsutra.guru",
        "position": "bottom-right",
    }
}

```

26. user (2026-06-21T07:40:30.562Z)

```

1     import cv2
2     import time
3     import random

```

```

4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")
31                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33             final_segments = self._detect_motion_segments(cap, max_frames)
34             segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
35
36             return Success(value=segments_data)
37
38         def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
39             """Run the pixel-motion heuristic over ``cap`` and return the detected
40             (start, end) play segments.
41
42             Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
43             thresholds it into raw segments, then merges dead-time gaps and filters
44             out segments shorter than ``min_segment_duration``. ``cap`` is released
45             before returning. This is the detection half of ``process_video`` ? it
46             performs no DB writes and no fabrication.
47             """
48             fps = cap.get(cv2.CAP_PROP_FPS)
49             total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
50             duration = total_frames / fps if fps > 0 else 0
51
52             # Sub-sample settings (5 FPS for analysis)
53             analysis_fps = 5.0
54             frame_interval = max(1, int(fps / analysis_fps))
55
56             idx_cfg = self.config.get("indexing", {})
57             motion_threshold = idx_cfg.get("motion_threshold", 0.08)
58             min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
59             dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
60
61             prev_gray = None
62             motion_signals = []
63             timestamps = []

```

```

65
66     frame_idx = 0
67     processed_count = 0
68     t_start = time.time()
69     # Emit a progress line roughly every 5% of the video so long runs are
70     # observable instead of silent (a full 1080p match is ~tens of minutes).
71     progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72     next_progress = progress_step
73     logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74               f"({duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75     while True:
76         # Skip decoding for intermediate frames using cap.grab() for high
performance
77         for _ in range(frame_interval - 1):
78             if not cap.grab():
79                 break
80             frame_idx += 1
81
82         ret, frame = cap.read()
83         if not ret:
84             break
85
86         # Process frame
87         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
88         # Apply Gaussian Blur to smooth noise
89         gray = cv2.GaussianBlur(gray, (21, 21), 0)
90
91         t = frame_idx / fps
92         timestamps.append(t)
93
94         if prev_gray is not None:
95             # Frame absolute difference (highly optimized motion tracking)
96             diff = cv2.absdiff(prev_gray, gray)
97             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
98
99             # Dilate to fill gaps
100            thresh = cv2.dilate(thresh, None, iterations=2)
101
102            # Motion ratio
103            motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
104            motion_signals.append(motion_ratio)
105        else:
106            motion_signals.append(0.0)
107
108        prev_gray = gray
109        frame_idx += 1
110        processed_count += 1
111
112        if progress_step and frame_idx >= next_progress:
113            pct = 100.0 * frame_idx / total_frames
114            elapsed = time.time() - t_start
115            eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
116            logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
117                      f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
118            next_progress += progress_step
119
120        if max_frames is not None and processed_count >= max_frames:
121            logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
122            break
123

```

```

124         if frame_idx >= total_frames:
125             break
126
127     cap.release()
128
129     # Smooth signal using a moving average
130     window_size = 5
131     smoothed = []
132     for i in range(len(motion_signals)):
133         start = max(0, i - window_size // 2)
134         end = min(len(motion_signals), i + window_size // 2 + 1)
135         smoothed.append(sum(motion_signals[start:end]) / (end - start))
136
137     # Detect active play boundaries
138     in_segment = False
139     segment_start = None
140     raw_segments: List[Tuple[float, float]] = []
141
142     for i, t in enumerate(timestamps):
143         signal = smoothed[i]
144         if signal > motion_threshold:
145             if not in_segment:
146                 in_segment = True
147                 segment_start = t
148             else:
149                 if in_segment:
150                     in_segment = False
151                     raw_segments.append((segment_start, t))
152
153     # Handle video ending while in active segment
154     if in_segment:
155         raw_segments.append((segment_start, timestamps[-1]))
156
157     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
158     merged_segments: List[Tuple[float, float]] = []
159     if raw_segments:
160         curr_start, curr_end = raw_segments[0]
161         for next_start, next_end in raw_segments[1:]:
162             if next_start - curr_end < dead_time_merge_gap:
163                 # Merge
164                 curr_end = next_end
165             else:
166                 merged_segments.append((curr_start, curr_end))
167                 curr_start, curr_end = next_start, next_end
168         merged_segments.append((curr_start, curr_end))
169
170     # Post-Processing: 2. Filter segments shorter than min_segment_duration
171     final_segments: List[Tuple[float, float]] = [
172         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
173     ]
174
175     return final_segments
176
177 def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
178                             player_pool: List[str]) -> List[Dict[str, Any]]:
179     """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181     For each detected segment this assigns a random server/receiver, fabricates
182     rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183     with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184     Returns the per-segment dicts handed back to the caller. The ``random``
185     call order here is load-bearing for reproducibility and must not change.
186     """
187     # Populate SQLite DB with segments and metadata events

```

```

188         segments_data = []
189     for start, end in final_segments:
190         # Assign Server and Receiver randomly from pool
191         server, receiver = random.sample(player_pool, 2)
192
193         # Formulate dynamic rich tags
194         dur = end - start
195         metadata = {
196             "shot_count": random.randint(4, 25),
197             "intensity": "high" if dur > 12 else "medium",
198             "avg_speed_kmh": round(random.uniform(40, 85), 1)
199         }
200
201         segment_id = self.db.add_play_segment(video_id, start, end, server,
202 receiver, metadata)
203
204         if segment_id:
205             # Add sub-segment events
206             # A: Serve Event (always at start)
207             serve_foul = random.random() < 0.15 # 15% chance of serve foul
208             self.db.add_event(
209                 segment_id=segment_id,
210                 timestamp=start + 0.5,
211                 event_type="serve",
212                 player=server,
213                 details={"status": "foul" if serve_foul else "valid", "type":
214 "low_serve" if random.random() > 0.5 else "high_serve"}
215             )
216
217             # B: Smash Events (1 to 3 random smashes per segment)
218             smashes_count = random.randint(0, 3)
219             for _ in range(smashes_count):
220                 smash_time = random.uniform(start + 1.5, end - 1.0)
221                 smasher = random.choice([server, receiver])
222                 self.db.add_event(
223                     segment_id=segment_id,
224                     timestamp=round(smash_time, 2),
225                     event_type="smash",
226                     player=smasher,
227                     details={"speed_kmh": round(random.uniform(120, 250), 1)}
228                 )
229
230             # C: Segment Termination Event (foul/winner)
231             foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
232             is_foul = random.random() > 0.4
233             error_player = random.choice([server, receiver])
234
235             self.db.add_event(
236                 segment_id=segment_id,
237                 timestamp=end - 0.2,
238                 event_type="foul" if is_foul else "winner",
239                 player=error_player,
240                 details={"type": random.choice(foul_types)} if is_foul else
241 {"shot_type": "drop" if random.random() > 0.5 else "smash"}
242             )
243
244             # Store segment representation to return to caller
245             segments_data.append({
246                 "id": segment_id,
247                 "start_time": start,
248                 "end_time": end,
249                 "duration": dur,
250                 "server": server,
251                 "receiver": receiver,
252                 "metadata": metadata

```

```

250         })
251
252         return segments_data
253
254     # Register the segmenter
255     segmenter_registry.register(MotionPlaySegmenter)
256

```

27. user (2026-06-21T07:40:31.591Z)

```

1     # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3     > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4     > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6     > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7     > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9     ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
15
16     Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole
loop on a single box, **honestly, behind a gate**.
20
21     **Headline CUJ (owner's words):** a **hosted website** where I pick an input ? **a
Google Drive file, a bucket file (`s3:///`gs://`), or a local upload** ? it lands in the
**bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host**
running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**
22
23     **Outcome ("good")** an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host
later** (native WASB) with **no UI fork**.
24

```

```

25     **Read to get current:** `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied **single origin**; engine stays `127.0.0.1:8000` | `[verified
main.py:638]`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | **Edge auth**, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32     | D3 | MVP drives the **in-process FastAPI worker** loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33     | D4 | Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim** | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34     | D5 | Console lives in the khelsutra **web/** React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35     | D6 | Compute placement is a **setting**, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36     | D7 | **Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
**stub** today ? scoped honestly ($4.2) |
37     | D8 | GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38     | D9 | **Observability is first-class** ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39     | D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop
app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the
tool | Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`** ; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41     ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42     - **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-
wiring only**, no engine change.
43     - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly **one job at a time** ; the DB `jobs` table is the
clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
**separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1
to serialize) ? it is **not** already 1.
44     - **Storage:** `storage.backend` default = `local` `[verified models.py:298]`;
`LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
local**. URI scheme `parse_uri` `[verified ref.py]`: bare/`local://` ? local; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub
(emulated only)**. Credentials never in config ? boto3 default chain / `~/aws` `[verified
s3.py]`.
45     - **Compute seam:** `indexing.detector_impl` ? `{`auto`=WSL, `native`=Linux-native,
`stub`=CPU mock}; default `auto` `[verified models.py:129]`, resolved in
`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner` : ** `native_wasb_runner.py` **exists** `[verified]`;
**`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46     - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +

```



```

--set indexing.skip_ai_handoff=true` + `storage.s3`.
47 - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` is **expected but does not exist** `[HOSTING_PLAN.md:42]`.
48 - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.
49 - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51 ## 4. Design / architecture
52
53 ### 4.1 Topology & request path
54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
127.0.0.1:8000** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56 ```
57 PUBLIC INTERNET
58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60 ?
61 ?????????????????????????????????????????????????????????????????????????????????
62 ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?
63 ? (2) reverse proxy / static host (Caddy/cloudflared) ?
64 ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?
65 ? ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66 ? ? ?
67 ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68 ? control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69 ? data: uploads.py chunked upload · /api/compile · /api/highlights ?
70 ? ? ?
71 ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72 ? ? ?
73 ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet] ?
74 ? native WASB (GPU host) · auto (owner Win+WSL) ?
75 ?????????????????????????????????????????????????????????????????????????????????
76 RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77 ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3:///` ?
outputs/<vid>/ back
78 (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79 ```
80
81 ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82 All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a
`StorageRef` (scale):
83
84 | Source | Path today | Status |
85 |---|---|---|
86 | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3:///`/`gcs:///`) | **wire client only** (endpoints exist)
|
87 | **Bucket URI** (`s3:///`, `gs:///`) | the worker already fetches `--video-uri` ? local ?
pipeline `[verified worker/___main__.py:98]`. From the UI, a small **ingest-to-process** step
(fetch the ref ? `/api/process`, or enqueue the worker) is **net-new** | **design** (storage
layer ready) |

```

```

88 | **Google Drive** (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90 ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91 | Placement | Runs | Needs | Real/mock | Honest caveat |
92 |---|---|---|---|---|
93 | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy**; **no per-video cost ceiling** today |
94 | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95 | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96 | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97 | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98 | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100 **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
102 ### 4.4 Auth / exposure (non-negotiable invariant)
103 No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
set `security.allowed_video_root`; rate-limit; per-user quota.
104
105 ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106 - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.
107 - **S2 Progress** ? 3s poll of the free-text stage
(`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-
failure terminal state.
108 - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`):
honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
**Per-rally is ONE screen of the console.**
109 - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
**reload-survival**.
110 - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112 ### 4.6 Reuse map
113 **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
bridge.
114
115 ## 5. Observability ? logging & monitoring `[required, first-class]`
116 Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously,
with a full log-analysis pass across every new code path.** The engine already uses a
centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
path.

```

```

117
118     - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`)
**plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and
into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ?
GPU host. Log it on every line.
119     - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry
/ result / error ** ? not just "it ran". The SPA logs client-side errors + failed fetches with
the same correlation id.
120     - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed
chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must
**never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
121     - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a
thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a
ledger exists) for the engine-mode banner + alerting.
122     - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is
**ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the
instance.
123     - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes
a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay**
($8 Layers 1?3). Ties [[feedback-launch-quality-bar]].
124
125     ## 6. Risk table
126     | Risk | Today | Mitigation |
127     |---|---|---|
128     | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall,
gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside
|
129     | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes
them |
130     | Retention | `output`/`+`uploads/` originals **never deleted** | sweep / keep-proxy-only
before any non-owner exposure |
131     | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap;
`ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132     | Native WASB unverified | partially wired; owner-parity-gated | treat as not-
production-verified until owner byte-parity on a real GPU box |
133     | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist |
coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD,
explicit `git add`** |
134     | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require
Drive?bucket pre-step until real |
135     | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136     | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137     | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive
/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139     ## 7. Honest limits ? what this does NOT do
140     Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142     ## 8. CUJs & live-test plan `[required]`
143     **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144

```

```

145     **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146     - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147     - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148     - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149     - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).
150     - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg`
behavior; a multi-GB browser upload. Reported back into $9, stated plainly as **not verified**
in the offline suite.
151
152     ## 9. Deliverables & progress tracker ? **source of truth**
153     Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR
ships its tests + meaningful logs ($5).
154
155     | ID | Deliverable | Depends on | Gated? | Status | PR |
156     |----|-----|-----|-----|-----|----|
157     | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? |
(this PR) |
158     | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ? |
? |
159     | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key
(P1) | ? | owner | ? | ? |
160     | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ? |
|
161     | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162     | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163     | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164     | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ? |
? |
165     | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) |
? | coord | ? | ? |
166     | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |
167     | T10 | Observability pass: correlation id end-to-end + structured logs + metrics ($5) |
T4 | no | ? | ? |
168     | T11 | Retention sweep for `output`/`+`uploads/` | ? | owner | ? | ? |
169     | T12 | Ingest from **bucket URI** (`s3://`/`gs://`) wiring | T4 | coord | ? | ? |
170     | T13 | **GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner
byte-parity (P4) | branch | ? owner | ? | ? |
171     | T14 | *(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |
172
173     **Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the
start** ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does **not** block on it (ships on Gemini).
174
175     ## 10. Open questions ? owner / external
176     **Blocking gates:** (1) **edge-auth** ? Caddy basicauth (self-contained) vs

```

```

cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention
defaults** ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4)
**GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.
177     **Nice-to-knows:** console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179     ## 11. Definition of done
180     - [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via
the ingest menu.
181     - [ ] Raw `:8000`/`:900x` unreachable from outside (**tested**, not assumed).
182     - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183     - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184     - [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via
`/api/videos` + `/api/reels`.
185     - [ ] **Observability:** correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186     - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187     - [ ] CI green (L2 contract + L4 regression); coverage ? floor; $9 tracker all
??-resolved.
188
189     ## 12. References
190     **Engine code `[verified this session]`:**
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
`PORTABILITY.md`, `web/README.md`.
191
192     ---
193     ### Changelog
194     - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs
(FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor
re-verified against source before writing (catching the native-WASB partial-wiring +
`ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),
multi-provider GPU, and first-class Observability directives.
195

```

28. assistant (2026-06-21T07:40:35.499Z)

This design doc is extremely thorough and was written this session against the same source. It confirms most of my findings. Let me read the remaining pieces: the config models (storage defaults, detector_impl default), the PER_RALLY_SELECTION doc, the trajectory_hybrid runner resolution, the gemini detector availability check, and verify what the SPA components are and whether there's any ingest UI. Let me also check the SPA's full component/structure and the static app.js the doc references.

29. user (2026-06-21T07:40:36.911Z)

```

1     # Interactive per-rally selection ? reel builder (RallyPicker)
2
3     > **Status:** `IN PROGRESS` ? **MVP / M0 (P1?P5) COMPLETE 2026-06-20** (pick ? query ?
select ? compile ? download, on existing engine endpoints; verified e2e against a stub engine,

```

GCP-GPU validation next). Design ****APPROVED** by owner 2026-06-20**. Downstream milestones M1?M5 (multi-video auth, per-rally preview endpoint, corpus loop, localized NL) remain owner-gated. .
****Owner:**** Avi . ****Created:**** 2026-06-20 . ****Last updated:**** 2026-06-20

4 > ****Lifecycle:**** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)

5 > ****Tracking anchors:**** \$7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in `NEXT\_STEPS.md` + memory `current-state` once work starts.

6 > ****Relation:**** second major tracked work item in the `khelsutra` repo (peer of `REQUEST\_ACCESS\_FORM.md`); mirrors the engine repo's `PROJECT\_DOC\_TEMPLATE` discipline. Primarily realizes alpha screen ****A5 Highlights**** (engine repo `UI\_PROPOSAL.md:47`, exec-plan ****M4**** `UI\_ALPHA\_EXECUTION\_PLAN.md:39`); its correction-loop extension realizes ****A4 Rally Review**** (`UI\_PROPOSAL.md:46`, exec-plan ****M3**** `:38`). Consumes the engine's `/api/query` + `/api/compile` contract.

7 > ****Honesty note:**** every engine-fact claim is tagged `[verified]` (read against engine code this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not legal/financial advice.

8

9 ---

10

11 ## 0. TL;DR

12 Build the ****pick-rallies ? reel**** surface as a ****hybrid****: a selectable rally ****list/grid**** is the workhorse; a ****deterministic query bar**** ("rallies over 10s, longest first") pre-selects into the ****same**** selection set; a sticky footer compiles the selection into one downloadable MP4. The entire MVP runs on ****existing engine endpoints**** ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights` `[verified backend/main.py:331-340, 342-396, 307-328]` ? with ****zero new engine intelligence and zero new endpoints****. The single most important caveat: queries and chips are scoped to ****length / count / order / shot-count only****; shot-type, player, and score are ****roadmap-only**** (no detector, no column) and must not be faked. The biggest UX-vs-effort tension: ****no per-rally preview is possible today**** ? only the ****final compiled reel**** can stream ? so MVP selects on text metadata, and visual preview/timeline is a deliberate later milestone.

13

14 ## 1. Problem & goal

15 Today the alpha can detect rallies and stitch a reel, but reel-building is either a ****silent `stitching.min\_shot\_count` threshold**** `[verified backend/main.py:362-383]` or a static checkbox list whose data plumbing is even broken ? the bundled static UI reads `data.rallies` while the API returns `play\_segments`, so it renders nothing `[verified backend/api/static/app.js:344 vs backend/main.py:340]`. The owner's framing is ****dynamic per-video selection, not a fixed threshold****: a coach should say "give me the long rallies from this session" and get exactly those, then fine-tune by hand.

16

17 ****The concrete outcome ("good")****: A coach uploads a 40-minute session; the engine finds 60 rallies. They type ****"top 15 longest"**, eyeball the auto-selected cards, untick two warm-up rallies, name it "Tuesday ? long points"**, and download a ~6-minute reel ? without learning any threshold knob.**

18

19 ****Goal****: the smallest ****honest**** surface that turns engine-detected rallies into a user-curated, downloadable reel, with both ****fast**** (query) and ****precise**** (checkbox) selection sharing ****one**** state.

20

21 ****Non-goals (alpha)****: no conversational LLM (`UI\_PROPOSAL.md:66` defers it); no shot-type/player/score filters (no data exists); no per-rally video preview while it requires unbuilt streaming; no multi-tenant auth (single-box alpha).

22

23 ## 2. Decisions locked

#	Decision	Source / date	Implication
D1	Dominant surface = selectable rally **list/grid**	Synthesis 2026-06-20 (Grid scored 15, fit-to-data 4)	Scales to long matches via bulk+filter; shows honest fields only
D2	Fast entry = **deterministic** client-side `parseQuery()` (no LLM)	[verified UI_PROPOSAL.md:66]	conversational QA deferred Predictable, unit-testable, offline, zero API cost
D3	**One** shared `Set<segment_id>`, mutated by query + checkbox + (future) timeline	Synthesis 2026-06-20	No second compile path can diverge from what the user sees
D4	Compile via existing `POST /api/compile {video_id, segment_ids, output_filename}`		

```

| `[verified backend/main.py:342-396]` | Selection order is harmless ? stitcher
`_merge_overlapping` de-dupes `[verified compiler.py:80-105, 311-317]` |
30 | D5 | Query grammar (alpha) = length, count, order, **shot_count only** | `[verified
database.py:122-135, gemini.py:470-485]` | Only fields the payload actually returns; grey out
the rest |
31 | D6 | MVP backend = **zero new endpoints** | `[verified]` reuse map ($4) | Ships the
full loop on the already-shipped contract |
32 | D7 | Replace the silent `min_shot_count` cut with **explicit user selection** | Owner
framing + `[verified main.py:362-383]` | UI *warns* about the floor; it never hides why a rally
vanished |
33 | D8 | **Default selection = ON** (curate-by-removal) | Owner 2026-06-20 | Rallies load
all-selected (matches today's "all rallies" reel); user removes warm-ups. Footer/empty UX
assumes a non-empty start |
34 | D9 | **Preview-before-select is NOT a ship-blocker** for the MVP | Owner 2026-06-20 |
MVP selects on text metadata + previews the *final compiled reel*; the source/clip-stream
endpoint (per-rally preview) is the next milestone (P9), not MVP |
35 | D10 | **MVP is job-tethered** (carry `result.video_id`); no `GET /api/videos` | Owner
2026-06-20 | Smallest honest slice; multi-video home + identity scoping is P8 (gated) |
36 | D11 | `min_shot_count`: **warn, do NOT override** in the MVP | Owner 2026-06-20 | The
UI surfaces the floor; an explicit-selection override of the compile default is deferred (would
trigger the Launch Report convention) |
37
38 ## 3. Foundation ? what the engine gives us today `[verified this session]`
39 The whole design rests on the engine's real, already-shipped contract. Read against the
engine repo (`badminton-highlight-indexer`):
40
41 - **The rally store is `play_segments`** ? columns `id, video_id, start_time, end_time,
duration, server, receiver, metadata` `[verified database.py:124-135]`. `id` is the integer the
UI selects and passes to compile; `start_time/end_time/duration` are REAL seconds (duration
computed at insert).
42 - **Listing rallies = `POST /api/query {video_id}` ? `{play_segments:[...]}`** ? the
*only* rally-list path today; it JSON-parses `metadata` and joins `events` `[verified
main.py:331-340, database.py:427-475]`.
43 - **Building a reel = `POST /api/compile {video_id, segment_ids:[int],
output_filename}`** ? loads `videos.filepath`, filters all segments to the requested
`segment_ids`, applies the `min_shot_count` floor, extracts `time_pairs=[(start,end)]`, and
calls `VideoCompiler.compile_highlights(raw_path, segments, name)` `[verified main.py:342-396,
compiler.py:303]`, which **merges overlaps** then per-clip re-encodes ? concat ? one MP4
`[verified compiler.py:311-317]`. Selection order/duplicates are therefore safe.
44 - **Downloading = `GET /api/highlights/{video_id}/{filename}`** ? streams the compiled
MP4. Compile returns a **server-local filepath, NOT a URL** `[verified main.py:394]`, so the SPA
constructs this download URL itself. The compiled reel is the **only streamable video that
exists today** `[verified main.py:307-328]`.
45 - **The floor knob is readable** ? `GET /api/config` exposes `stitching.min_shot_count`
`[verified main.py:121-123]`, so the UI can warn before a selected rally is silently dropped.
46 - **Job entry** ? `GET /api/jobs/{job_id}` returns `result.video_id` `[verified
main.py:274-293]`, which is how the MVP reaches a video without a (nonexistent) list-videos
endpoint.
47
48 **Honest fields vs fabricated/roadmap (critical for the UI):**
49 - **Honest today** (default `default_segmenter: "gemini"`, per `config.json`
`[verified]`): `start_time`, `end_time`, `duration`, and `metadata.shot_count` (Gemini's
`shuttle_exchange_count`, fallback `max(3, int(dur/0.8))`) + `metadata.shot_count_confidence` (a
**string**, often `"Unknown"`) `[verified gemini.py:470-485]`. On the gemini path
`server`/`receiver` are written `None` ("no fabricated data") `[verified gemini.py:480-487]`.
50 - **Fabricated ? do NOT surface:** the `motion` demo segmenter fills `server`/`receiver`
via `random.sample(...)` , a random `shot_count`, and random `events` (serve/smash/foul/winner)
`[verified motion.py:191-198]`. `query` returns these columns + `events`, but they are not
ground truth and must stay hidden.
51 - **Roadmap ? no column, no detector:** shot-type per shot, player identity, match/point
score, game/set grouping. None exist anywhere in the schema; do not consume them `[verified
database.py whole-schema 122-148]`.
52 - **Correction-loop columns exist but are dead:**
`candidate_segments.human_label/notes/confirmed_segment_id` are defined but no endpoint writes
them `[verified database.py:166-168]` ? that's the M4 corpus extension (engine PR-4/B6,

```

```

`UI_ALPHA_EXECUTION_PLAN.md`).
53
54     ## 4. Design / architecture
55
56     **Reuse map (existing ? `[verified]`, no change for MVP):** `POST /api/query` (list) ?
`POST /api/compile` (stitch) ? `GET /api/highlights/{video_id}/{filename}` (download); `GET
/api/config` (floor) ; `GET /api/jobs/{id}` (carry `result.video_id`). **All net-new code is the
SPA** in `khelsutra` `web/` (Vite + React + TS + Tailwind, per `web/README.md`), plus a small
set of *optional* real-gap engine enablers (§4.3) ? none required for MVP.
57
58     **4.1 Components (net-new SPA)**
59     - **`QueryBuilder + `parseQuery()`** ? a *pure, deterministic, unit-tested* string ?
`{filter, sort}` compiler over `duration` / `ordinal` / `shot_count`. Renders a **parsed-pill**
("filter: duration > 10s · sort: duration desc") so the interpretation is transparent and hand-
correctable; greys out shot-type/player/score tokens with an inline "not available yet". No LLM,
no network call.
60     - **`RallyList/Grid`** ? one card per `play_segment`: ordinal (by `start_time`), `mm:ss`
start, duration (`end?start`), and a shot-count + confidence chip **only when
`metadata.shot_count` is present** (absent on detector paths that don't write it). Checkbox per
card; the thumbnail area is a placeholder (no endpoint yet).
61     - **`useRallySelection()` hook** ? owns `selectedIds:Set<number>`, `displayOrder`,
`apply(queryResult)`, `toggle(id)`, and bulk Select-all / Clear / Invert. Unit-tested (matches
the exec-plan's "API client + review-state logic" bar).
62     - **`SelectionFooter` (sticky)** ? live "N clips / total M:SS"; a **`min_shot_count`
warning** when a selected rally would be dropped at compile (read from `GET /api/config`); reel-
name input; one **`Build`** button.
63     - **`CompileResult`** ? `[` preview of the *compiled
reel* + a download link.
64     - **`TypedApiClient`** ? `query`, `compile`, `buildDownloadUrl`; the typed seam to the
engine.
65
66     **4.2 Data flow** `[verified anchors inline]`
67     Gemini segmenter (process time) writes `play_segments {id, start/end_time, duration,
metadata.shot_count/confidence/detector}` `[gemini.py:461-498]`
68     ? SPA `POST /api/query {video_id}` ? `{play_segments:[...]}` `[main.py:331-340]`
(**fix** the static UI's `data.rallies` read ? `play_segments` `[app.js:344]`)
69     ? normalize to `RallyCard[]` + one shared `selectedIds:Set`;
`server`/`receiver`/`events` are present in the payload but motion-only fabrications ? **not
surfaced** `[motion.py:191-198]`
70     ? all input modes mutate the **same** `selectedIds`: (a) `parseQuery()` predicate over
duration/ordinal/shot_count; (b) checkbox toggle + bulk ops; (c) *(post-MVP)* timeline band
71     ? footer recomputes count + ?duration **client-side**, and warns for any selected rally
with `shot_count < stitching.min_shot_count` `[main.py:362-383]`
72     ? **`Build`** ? `POST /api/compile {video_id, segment_ids: [...selectedIds],
output_filename}` ? stitcher de-dupes via `_merge_overlapping` `[compiler.py:311-317]`, cuts
`time_pairs` from `videos.filepath`, re-encodes per clip ? concat ? one MP4 `[main.py:386-393,
compiler.py:303]`
73     ? compile returns a **server-local filepath, not a URL** `[main.py:394]` ? SPA builds
`GET /api/highlights/{video_id}/{output_filename}` to preview in `` + download
`[main.py:307-328]`.
74
75     **4.3 Engine API additions (none for MVP; tagged against real gaps)**
76     | Endpoint | Purpose | Status |
77     |---|---|---|
78     | `POST /api/query` | List rallies (only path) ? fix SPA to read `play_segments` |
**exists** |
79     | `POST /api/compile` | Stitch selected `segment_ids` ? MP4 | **exists** |
80     | `GET /api/highlights/{video_id}/{filename}` | Stream/download compiled reel |
**exists** |
81     | `GET /api/config` | Read `stitching.min_shot_count` for the warning | **exists** |
82     | `GET /api/jobs/{job_id}` | Carry `result.video_id` into the picker | **exists** |
83     | `POST /api/compile` ? add `download_url` | Return `/api/highlights/...` so SPA stops
hand-building it (additive, low-risk; today returns only a filepath `main.py:394`) | **modify**
|
84     | `GET /api/health` | Engine-offline banner ping the SPA is specified to use
](GET)
```



```

(`HOSTING_PLAN.md:42`) | **new** |
85 | `GET /api/videos/{video_id}/rallies` | RESTful/cacheable alias returning the same
  | {play_segments}` shape | **new** |
86 | `GET /api/videos` | List a user's videos for a Match picker (only `get_video(id)`
  | exists, `database.py:418`) ? needs DB enumerate + identity scoping | **new** |
87 | `GET /api/videos/{video_id}/source` (Range) *or* `/clip?start=&end=` | Stream
  | source/per-span video so a rally previews before selection ? neither exists | **new** |
88 | `GET /api/videos/{video_id}/rallies/{segment_id}/thumbnail` | Per-rally poster still
  | (no thumbnail column; WASB frames are transient) | **new** |
89 | `PATCH /api/segments/{id}` | Persist accept/reject/nudge ?
  | `candidate_segments.human_label/notes/...` (columns exist, unwritten ? corpus loop) | **new** |
90
91 *** Dead-ends / do-NOT:*** never surface `server`/`receiver`/`events` (motion-only
  | randoms `[motion.py:191-198]`); never consume shot-type/player/score (no data); never render a
  | confidence **meter** ? `shot_count_confidence` is a Gemini *string* (often "Unknown") and
  | `play_segments` has **no** numeric confidence column.
92
93 **4.4 Observability (first-class, per `[memory: feedback-launch-quality-bar]`)** Every
  | P3 user action emits a **structured JSON log line** (`web/src/lib/log.ts`) so a journey is
  | traceable from text to selection ? the SPA-side "runlog". Each query application generates **one
  | `cid`** shared by every line of that journey; the future `/api/compile` call (P5) reuses the
  | same `cid` to thread the trace into the engine. Events (all on the P3 path):
94
95 | event | level | when | key fields |
96 |---|---|---|---|
97 | `query.parse` | info | every apply attempt | `cid`, `source` (typed/example),
  | `recognized`, `filters`, `sort`, `limit`, `unavailable` ? the exact interpretation |
98 | `query.unavailable_request` | info | the query names a shot-type/player/score | `cid`,
  | `raw`, `unavailable[]` ? **demand signal** for roadmap features (what users ask for that we
  | can't yet do) |
99 | `query.skip` | info | nothing actionable parsed | `cid`, `raw`, `reason` |
100 | `query.apply` | info | a recognized query is applied | `cid`, counts
  | (`total`/`selected`), `selected_ids`, `dropped_unavailable` |
101 | `query.empty_result` | **warn** | recognized query matches **zero** rallies | `cid` +
  | full interpretation ? a looks-empty grid is **never silent** |
102 | `query.apply_failed` | **error** | the (pure) parser/selector unexpectedly throws |
  | `cid`, `raw`, `error` ? loud, never swallowed |
103 | `selection.bulk` | info | Select-all / Clear / Invert | `cid`, `op`, `result_count` |
104 | `rally.toggle` | debug | a card is added/removed | `id`, `action` |
105
106 No silent drops: unsupported concepts (`query.unavailable_request` +
  | `dropped_unavailable`) and empty results (`query.empty_result` warn) are always logged. Verified
  | live in-browser (events fire with correct `cid`/fields, zero console errors).
107
108 ## 5. Risk table
109 | Risk | Severity | Mitigation |
110 |---|---|---|
111 | **No source/clip/frame/thumbnail endpoint** ? only compiled reels stream
  | `[main.py:307-328]` | High (UX) | MVP selects on text metadata + previews the *final* reel;
  | preview/timeline deferred to M3 behind a new endpoint |
112 | `min_shot_count` silently drops selected rallies `[main.py:362-383]` | High (looks
  | broken) | Footer warns *before* Build; D7 makes selection explicit; an override is an owner gate
  | (Launch-Report-worthy `[memory: launch-report-convention]`) |
113 | No match-duration field ? a timeline must derive `total = max(end_time)`, mis-
  | rendering pre/post dead air | Med | Timeline deferred to M3; never over-promise it as a true
  | scrubber until source streams |
114 | `shot_count` is an *estimate* (`shuttle_exchange_count` w/ `max(3,int(dur/0.8))`
  | fallback) `[gemini.py:470-485]` | Med | Label "approx."; confidence shown as the raw string, not
  | a meter |
115 | No auth / user scoping anywhere (CORS `*`, `videos.user_id` never set/filtered) | High
  | for multi-tester | Fine for single-box local alpha; a ship-blocker for any shared host ? gate
  | `GET /api/videos` behind PR-3 scoping |
116 | compile returns a filepath not a URL `[main.py:394]` | Low | Client builds the URL for
  | MVP; `download_url` field (M1) removes the brittleness |
117 | Surface sprawl (query + grid + future timeline + future correction) | Med | Deliberate

```

IA: query+grid = one screen; correction = separate ? corpus screen (owner call, S8) |

118

119 ## 6. Honest limits ? what this does NOT do

120 A green SPA build + passing `parseQuery`/selection unit tests prove the **picker logic only** ? **not** that the engine's rally boundaries are correct (the measured **rally-END** weakness is a separate quality track, unaffected by this UI). "Conversational" here means a **deterministic four-dimension parser**, not natural-language understanding. Nothing is implemented in `web/` yet ? this is `[design]`. The feature curates *what the engine already found*; it cannot surface a rally the detector missed, nor split/merge a mis-bounded one (that's the M4 correction loop).

121

122 ## 7. Deliverables & progress tracker ? **source of truth**

123 Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking. CI (`npm test`) must stay green; merge to `master` auto-deploys the site (docs/`web/` changes don't touch the live `site/` Worker).

124

	ID	Deliverable	Depends on	Gated?	Status	PR
	P0	This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers			?	No
[#11](https://github.com/avidullu/khelsutra/pull/11)	P1	SPA scaffold (Vite+React+TS+Tailwind) in `web/` + typed API client; dev proxy to engine `:8000`	P0	No	?	[#14](https://github.com/avidullu/khelsutra/pull/14)
	P2	RallyList/Grid (honest fields only) + `useRallySelection()` hook (unit-tested)	P1	No	?	[#21](https://github.com/avidullu/khelsutra/pull/21)
	P3	`parseQuery()` + parsed-pill + example chips (unit-tested; greys out unavailable tokens)	P2	No	?	(this PR)
	P4	SelectionFooter + `min_shot_count` warning (from `GET /api/config`)	P2	No	?	(this PR)
	P5	Build ? `POST /api/compile` ? client-built download URL + ` <video>` preview of the reel</video>	P2	No	?	(this PR)
	P6	*(engine repo)* `GET /api/health` + offline banner; `/api/compile` returns `download_url`			?	No
	P7	*(engine repo)* `GET /api/videos/{id}/rallies` RESTful alias			?	No
	P8	*(engine repo)* `GET /api/videos` + PR-3 identity scoping ? A1 Matches screen	P7	?	owner	?
	P9	*(engine repo)* source/clip stream + timeline + per-rally preview/thumbnail	P8	?	owner	?
	P10	*(engine repo)* `PATCH /api/segments/{id}` corpus loop (PR-4/B6) ? A4 Rally Review	P9	?	owner	?

138

139 **Milestone rollup:** **M0/MVP = P1?P5 ? COMPLETE (2026-06-20)** (pick?reel on existing API, zero new engine endpoints) · **M1 = P6?P7** (RESTful + reliability polish) · **M2 = P8** (multi-video home) · **M3 = P9** (per-rally preview/timeline) · **M4 = P10** (corpus correction loop) · **M5** = swap `parseQuery()` for localized NL behind the same seam (gated on shot/player data landing).

140

141 ## 8. Open questions ? owner / external

142 **Blocking gates ? ? RESOLVED 2026-06-20 (owner) ? see D8?D11 in S2:**

143 1. ~~Default selection ON vs OFF?~~ ? **ON / curate-by-removal** (D8).

144 2. ~~Is preview-before-select a ship-blocker?~~ ? **No** ; text-metadata MVP, preview is P9 (D9).

145 3. ~~Job-tethered vs multi-video MVP?~~ ? **Job-tethered** (D10).

146 4. ~~`min\_shot\_count` override?~~ ? **Warn, don't override** in MVP (D11).

147

148 **Still open (nice-to-knows, non-blocking):**

149 5. May user-facing copy call `parseQuery()` "conversational", given conversational QA is explicitly deferred (`UI\_PROPOSAL.md:66`)? Risk of over-promising NL. *Owner positioning/honesty call.* (Lean: avoid the word; show the parsed-pill instead.)*

150 6. One screen for query+grid (recommended) vs a separate ? corpus/correction screen for A4? *Owner positioning call.*

151

152 ## 9. Definition of done

153 - [] From a finished job, a user can list rallies, query/select, and download a reel ? on **existing** endpoints.

154 - [] Only honest fields (ordinal / duration / shot_count+confidence-when-present) are

shown; fabricated `server`/`receiver`/`events` are hidden.

155 - [x] `parseQuery()` covers length/count/order/shot_count, greys out unavailable tokens, shows a visible parsed-pill, and is unit-tested. *(P3)*

156 - [x] The P3 path is **observable**: structured, correlation-id'd logs for parse / apply / skip / empty / unavailable-request / manual-edit; unsupported concepts and zero-result queries are logged loudly, never dropped silently (\$4.4). *(P3)*

157 - [x] Selection hook is unit-tested; the `min\_shot\_count` floor is surfaced, not hidden. *(P2 hook · P4 floor warning ? `lib/floor.ts` mirrors `main.py:362-383`, exempts no-shot_count rallies, blocks Build when all-dropped)*

158 - [x] From a finished job, a user can list rallies, query/select, and download a reel ? on **existing** endpoints. *(P5 ? `?video=<id>` loads via `POST /api/query`; Build ? `POST /api/compile` ? `<video>` preview + download over `GET /api/highlights/...`; verified e2e against a stub engine)*

159 - [x] `npm run build` (and `npm test`) clean; the static UI's `data.rallies` bug is gone ? the SPA **supersedes** the static `app.js` for this flow (reads `play\_segments`).

160 - [x] S7 tracker + this DoD updated. *(Doc stays live for the owner-gated M1?M5 engine milestones; not archived ? only the MVP/M0 slice is complete.)*

161

162 **## 10. References**

163 **Engine code anchors** `[verified this session]`: **backend/main.py:121-123** (config), **:274-293** (jobs), **:307-328** (highlights download), **:331-340** (query), **:342-396** (compile + `min\_shot\_count` floor + filepath-not-URL); **backend/infrastructure/database.py:124-135** (play_segments), **:166-168** (dead correction columns), **:418** (no list-videos), **:427-475** (query read path + events join); **backend/pipeline/segmenters/gemini.py:461-498, 470-485** (honest shot_count, server/receiver=None); **backend/pipeline/segmenters/motion.py:191-198** (fabricated fields); **backend/pipeline/stitcher/compiler.py:80-105, 303, 311-317** (merge/de-dupe + entypoint); **backend/api/static/app.js:344** (`data.rallies` bug); **config.json** (`default\_segmenter: "gemini")`).

164 **Engine docs**: **UI_PROPOSAL.md:46-47, 66**; **UI_ALPHA_EXECUTION_PLAN.md:38-39**; **HOSTING_PLAN.md:42**; **OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55** (conversational seam built now, model punted). **This repo**: **web/README.md**, **REQUEST_ACCESS_FORM.md**.

165

166 ---

167 **### Changelog**

168 - 2026-06-20 ? Initial `DRAFT`. Consolidated three candidate designs (Timeline / Grid / RallyPicker) into the hybrid recommendation via a multi-agent design pass; every engine-fact anchor re-verified against source before writing.

169 - 2026-06-20 ? Owner **approved** the recommendations; locked D8?D11 (default ON · preview not a ship-blocker · job-tethered MVP · warn-don't-override the floor); status `DRAFT` ? IN PROGRESS`; P0 done.

170 - 2026-06-20 ? **UI**: lead with rally LENGTH, park shot-count as experimental** (owner). Rally cards emphasize duration (`m:ss long`); the shot-count chip is flagged amber "experimental" with a tooltip ("under development, may be inaccurate ? filter by length instead"); example chips are duration-led (dropped "at least 8 shots"); a note states "rally length is fully supported, shot-count is experimental". `parseQuery` still accepts shot-count queries (not broken) but they're de-featured. Public badge `P5`?`alpha`. A test pins that every example chip parses to a recognized, fully-available (non-shot-count) query. Suite 75 green.

171 - 2026-06-20 ? **P5 shipped** ? MVP / M0 COMPLETE.** `App` now loads a real session from `?video=<id>` (`POST /api/query`, default ALL-selected D8) with a mode banner (demo / loading / loaded N / error); **Build** ? `compileReel` (`POST /api/compile`) ? `CompileResult` previews the stitched reel in `<video>` over `GET /api/highlights/{video\_id}/{filename}` (client-built URL ? engine returns a filepath, not a URL) + a download link; `reelFilename()` slugifies to an engine-safe bare `.mp4`. Loading/building/done/error states, all traced (`rallies.load\_\*`, `build.start/done/failed`). Verified e2e against a stub engine: load 5 rallies ? real `api/config` floor warning ? Build ? `<video>` reached `readyState=4` (decoded a real mp4 through the proxy) + download link correct; all-below-floor selection keeps Build disabled (P4 guard prevents the 400). `reelFilename` unit-tested; suite **73 green**, typecheck + build clean. **Next**: GCP-GPU end-to-end validation (real video ? rallies ? reels).**

172 - 2026-06-20 ? **P4 shipped**: `SelectionFooter` (live summary + reel-name input + Build) with an **honest** `min\_shot\_count` warning ? `lib/floor.ts` mirrors the engine floor (`main.py:362-383`) EXACTLY: a rally is dropped only if it has a numeric `shot\_count` below the floor; rallies **without** `shot\_count` are exempt; **all-below** ? Build disabled** (the engine would 400). Config read from `GET /api/config` on mount (graceful offline). 8 floor unit tests; amber/red warning states verified live in-browser; suite **70 green**, typecheck + build clean.

173 - 2026-06-20 ? **P3 shipped**: deterministic `parseQuery()` (`web/src/lib/query.ts`)

over length/count/order/shot_count + `runQuery`/`sortRallies`/label helpers and the `QueryBar` (live parsed-pill, example chips, greyed "not available yet" for shot-type/player/score ? never faked). Writes the shared selection set via a new `useRallySelection().apply`; `RallyGrid` ordinals made stable so a re-sort doesn't renumber cards. **First-class observability (§4.4):** structured, correlation-id'd `query.\*` / `selection.\*` / `rally.toggle` runlog (loud `empty\_result`/`apply\_failed`; `unavailable\_request` demand signal). **Adversarial multi-agent review** (grammar/honesty/robustness/reuse) ran pre-merge ? 13 confirmed findings folded in, incl. 4 majors: an honesty leak (`at least 8 smashes` ? fake `length` ? 8s), a selection-wiping `top 0`, a unit-blind `first 30 seconds`, and self-contradictory pill labels (`first 10` shown as "top 10"). Live browser CUJ replayed (filter / top-N / honest-greying / untick / no-op / wipe-closed). Suite **62 green**, typecheck + build clean.

174

30. user (2026-06-21T07:40:38.684Z)

=== web src tree ===

```
src/App.tsx
src/api/client.test.ts
src/api/client.ts
src/api/types.ts
src/components/CompileResult.tsx
src/components/LanguageSwitcher.tsx
src/components/QueryBar.examples.test.ts
src/components/QueryBar.tsx
src/components/RallyGrid.tsx
src/components/SelectionFooter.tsx
src/hooks/useRallySelection.ts
src/i18n/hygiene.test.ts
src/i18n/i18n.test.tsx
src/i18n/index.ts
src/i18n/locales/en/common.json
src/i18n/locales/hi/common.json
src/i18n/locales/kn/common.json
src/index.css
src/lib/floor.test.ts
src/lib/floor.ts
src/lib/format.test.ts
src/lib/format.ts
src/lib/log.ts
src/lib/query.test.ts
src/lib/query.ts
src/lib/selection.test.ts
src/lib/selection.ts
src/main.tsx
src/test/setup.ts
```

=== package.json scripts+proxy ===

```
{
  "name": "khelsutra-web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer engine).",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit",
    "test": "vitest run"
  },
  "dependencies": {
    "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
    "@fontsource-variable/noto-sans-kannada": "^5.2.9",
    "i18next": "^26.3.1",
    "i18next-browser-languagedetector": "^8.2.1",
    "i18next-icu": "^2.4.3",
```

```

    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-i18next": "^17.0.8"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.0.0",
    "@testing-library/dom": "^10.4.1",
    "@testing-library/react": "^16.3.2",
    "@types/node": "^26.0.0",
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "jsdom": "^29.1.1",
    "tailwindcss": "^4.0.0",
    "typescript": "^5.7.0",
    "vite": "^6.0.0",
    "vitest": "^3.0.0"
  }
}
=== vite proxy ===
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
// proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
// (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local engine
// (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if needed.
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: {
      "/api": { target: "http://localhost:8000", changeOrigin: true },
    },
  },
  test: {
    // Default env is node (the pure lib/parser tests); the i18n render test opts into jsdom via
a    // `/// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
    environment: "node",
    include: ["src/**/*.test.{ts,tsx}"],
    // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files no-op.
    setupFiles: ["src/test/setup.ts"],
  },
});

```

31. user (2026-06-21T07:40:39.168Z)

```

110     cue_flags: dict[str, bool] = Field(default_factory=dict)
111     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
113     min_crossings: int = Field(default=0, ge=0)
114     # Served Gate B: when the person cue is on, drop windows with median in-court
players
115     # < this. 0 = OFF.
116     min_players: int = Field(default=0, ge=0)
117     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
118     court_polygon: Optional[list[list[float]]] = None
119     # Net line for the person two-sided-motion split (person features only ? the shuttle
120     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
121     net_axis: Literal["x", "y"] = "y"
122     net_position: float = Field(default=0.5, ge=0.0, le=1.0)

```

```

123
124
125     class IndexingConfig(BaseModel):
126         default_segmenter: str = "yolo_hybrid"
127         use_gpu: bool = True
128         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
129         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
130         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
131         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
132         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
133         detector_impl: str = "auto"
134         motion_threshold: float = 0.08
135         min_segment_duration: float = 3.0
136         dead_time_merge_gap: float = 2.0
137         chunk_duration: float = 90.0
138         chunk_overlap: float = 10.0
139         detector_model: str = "fasterrcnn_resnet50_fpn"
140         detector_score_threshold: float = 0.5
141         yolo_velocity_threshold: float = 0.15
142         yolo_frame_skip: int = 3
143         yolo_tracker: str = "bytetrack.yaml"
144         yolo_prefetch_workers: int = 2
145         min_action_window_duration: float = 2.0
146         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
147         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?)
148         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
149         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
150         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
151         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
152         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
153         max_window_duration: float = 6.0
154         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
155         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
156         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
157         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
158         # Only cuts (recall can't drop). OFF by default until measured/promoted.
159         window_hard_cap: bool = False
160         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
161         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
162         # [|?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
163         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
164         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
165         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [|?end| 4.96?3.31s (n=13).
The W1 cap
166         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
167         window_inactivity_end: float = 1.5
168         inactivity_rally_thresh: float = 0.20

```

169 inactivity_min_density: float = 0.30

32. assistant (2026-06-21T07:40:45.570Z)

I now have a very complete picture. The two design docs were written THIS session against the same source and are heavily `[verified]`-tagged. Let me verify a few last specifics: (1) the storage config defaults and whether gcs settings exist, (2) the gemini detector's availability/key check, (3) the trajectory_hybrid `\_resolve\_runner` (auto/native/stub seam), and (4) confirm the SPA has no ingest/process/upload wiring (grep the SPA for process/upload/job/capabilities). Let me also confirm `download\_url` is NOT consumed by the SPA.

33. user (2026-06-21T07:40:47.014Z)

```
280           # mode). Default None preserves the single-user local 'any local file' workflow.
281           allowed_video_root: Optional[str] = None
282
283           # --- Chunked upload (B2, PR-2) ---
284           # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285           # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286           # contain upload_dir or /api/process will reject the uploaded paths.
287           upload_dir: str = "output/uploads"
288           # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289           # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
290           max_upload_mb: int = 4096
291           max_part_mb: int = 95
292           allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295       class StorageConfig(BaseModel):
296           """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297           behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298           + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299           Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300           **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301           (boto3/google auth chains supply credentials). See ``backend/storage/``.
302           backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303           # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304           output_root: str = ""
305           # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
306           # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
307           # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
308           # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
309           # the legacy flat layout is used unchanged.
310           single_folder_per_video: bool = False
311           # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
312           workspace_root: str = ""
313           # Cloud namespace prefix under the root so per-video folders sit tidily beside
314           # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
315           workspace_prefix: str = "workspaces"
316           local: dict[str, Any] = Field(default_factory=dict)
317           s3: dict[str, Any] = Field(default_factory=dict)
318           gcs: dict[str, Any] = Field(default_factory=dict)
319           gdrive: dict[str, Any] = Field(default_factory=dict)
320
321
322       class DeploymentConfig(BaseModel):
```

```

323     """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
324     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
325
326     **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
327     behaves exactly as before this section existed. A profile is opt-in (``config.json``
328     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
329     *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
330     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
331     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
332
333     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
334
335     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
336     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
337     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
338     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
339     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""

```

34. user (2026-06-21T07:40:48.630Z)

No matches found

35. user (2026-06-21T07:40:49.409Z)

```

40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47         #: config section under `indexing` (velocity_threshold lives there), the
48         #: output subdir for trajectories, and the metadata detector-tag prefix.
49         family: str = ""
50         #: human-readable label used in log lines.
51         display_label: str = ""
52
53         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54             """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
55             raise NotImplementedError
56
57         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
58             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
59             native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
60             raise NotImplementedError(
61                 f"detector_impl='native' is not supported for the "
62                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65         def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:

```



```

66         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73         (the WSL runner) so the default production path is unchanged
74         (``docs/DECOUPLED_COMPUTE/``).
75         """
76         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77         if impl == "stub":
78             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                         self.display_label or "trajectory")
81             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82         if impl in ("native", "native_wasb", "wasb_native"):
83             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                         self.display_label or "trajectory")
85             return self._build_native_runner(idx_cfg)
86         return self._build_runner(idx_cfg)
87
88     @staticmethod
89     def _probe_fps_width(video_path: str) -> tuple:
90         """Return (fps, frame_width) for a video, with sensible fallbacks."""
91         cap = cv2.VideoCapture(video_path)
92         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94         cap.release()
95         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97     @staticmethod
98     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99         """Provider-reported exchange count, else a duration-based estimate.
100
101         One canonical implementation ? the twins had drifted (wasb relied on
102         ``int(None)`` raising into the fallback; same result, by accident).
103         """
104         shot_count = segment.get("shuttle_exchange_count")
105         if shot_count is None:
106             return max(3, int(duration / 0.8))
107         try:
108             return int(shot_count)
109         except (ValueError, TypeError):
110             return max(3, int(duration / 0.8))
111
112     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-----
113     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                frame_width: float, video_path: str,
115                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119         features). ``points`` are the whole-video trajectory points

```

```

(TrajectoryPoint)."""
120         return {
121             "peak_velocity": cw["peak_velocity"],
122             "mean_velocity": cw["mean_velocity"],
123             "active_frame_count": cw["active_frame_count"],
124             "track_id_count": cw["track_id_count"],
125         }
126
127     def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                           window_stats: List[Dict[str, Any]],
129                           idx_cfg: Dict[str, Any]) -> List[int]:
130         """Indices of the candidate windows that proceed to the AI handoff (and thus may
131         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133         Persisted candidates are NOT affected ? they remain the full superset for
offline
134         re-scoring."""

```

36. assistant (2026-06-21T07:40:54.293Z)

The grep confirms a critical SPA gap: the SPA source contains **zero** references to ``/api/process`, `/api/upload`, `/api/jobs`, `/api/capabilities`, `/api/videos`, `/api/reels`, `/api/health`, `job_id`, or `download_url`. The SPA only knows /query`, `/compile`, `/highlights`, `/config`. Let me do a couple final verifications: the gemini detector's availability/key requirement (for "what a real dry-run needs"), the stub_runner (CPU-only real-ish path), and whether /api/health` actually exists now (the doc said "expected but does not exist" ? but main.py:126 shows it DOES exist now). Let me confirm those.`

37. user (2026-06-21T07:40:58.008Z)

```

=== gemini availability/key ===
86:     api_key_env_var = f"{provider_name.upper()}_API_KEY"
87:     api_key = os.environ.get(api_key_env_var)
88:     if not api_key:
89:         from backend.pipeline.segmenters._keyhint import api_key_hint
91:         f"{api_key_env_var} environment variable is not set! "
93:         + api_key_hint(api_key_env_var)
98:     provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
501:     def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:

```

```

=== stub_runner head ===
"""CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".

```

A ```DetectorRunner``` that produces a deterministic ```Frame,Visibility,X,Y``` trajectory CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline (```wasb_hybrid`` / ``tracknet_hybrid```) can run end-to-end on a CPU-only box (CI, a CPU container, a laptop) for regression testing. The person/yolo path already CPU-falls-back (```use_gpu=False```); this closes the last GPU/WSL-only gap so the *entire* pipeline is runnable ? and regression-gateable ? without a GPU.

Two modes:

- * **replay** ? emit an existing ```Frame,Visibility,X,Y``` CSV verbatim (drive the pipeline with a committed/golden trajectory; deterministic + cross-OS).
- * **synth** (default) ? generate a deterministic synthetic trajectory of ```num_rallies``` in-flight bursts (visible + fast-moving frames) separated by dead gaps, so windowing yields a stable, known number of candidate windows.

This is the CPU seam the ```DetectorRunner``` ABC was built for (see ```base.py``` ? "Optional Linux-native path") and the **seed of the future ```LinuxNativeRunner```** (M3.5, ```docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md```). It is selected via ```indexing.detector_impl="stub"``` or the ```RALLY_DETECTOR_IMPL=stub``` env var; the real WSL runners stay the default, so production behaviour is unchanged.

```

import os
import csv as _csv
import shutil
import logging
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

```

```

from backend.pipeline.detectors.base import DetectorRunner

```

Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).

```

from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

```

```

logger = logging.getLogger(__name__)

```

```

@dataclass

```

```

class StubConfig:

```

```

    """Settings for the CPU stub runner (built from ``config.json`` -> ``indexing.stub``)."""
    fps: float = 30.0

```

```

=== does /api/health exist + git log of M1 branch ===

```

```

9c64ac5 docs(serving): mark $7 tracker M1 ? + changelog (PR #279)

```

```

368155f feat(serving): M1 ? deployment.profile + compute_target config (pure, additive, default-OFF)

```

```

f518e97 docs(serving): M0 APPROVED + lock D13-D15 (switchable profiles, Drive Northstar, consent-not-surprise) (#278)

```

```

49c082a docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness (M11) (#277)

```

```

f592281 docs: point I18N_PLAN + UI_PROPOSAL (+ DOC_STATUS) at the khelsutra UX/l10n project (#275)

```

```

f6f15ca docs: isolate field-ops/PMF + Roora into archives/; carve a no-ML docs/data_pipeline/ (#276)

```

```

03fa951 docs: resolve DOC_STATUS findings #1 (per-video supersede) + #2 (NEXT_STEPS refresh) (#274)

```

```

aa5e9db docs: add DOC_STATUS register + archival due-diligence convention (#273)

```

38. user (2026-06-21T07:40:58.367Z)

```

1      // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile succeeds,

```

```

2      // preview the reel inline (<video> over GET /api/highlights/...) and offer a download. The engine

```

```

3      // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL itself. On a

```

```

4      // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never swallowed.

```

```

5
6      import { useTranslation } from "react-i18next";

```

```

7
8      export type BuildState =
9          | { status: "idle" }
10         | { status: "building" }
11         | { status: "done"; url: string; filename: string }
12         | { status: "error"; error: string };

```

```

13
14     interface Props {
15         build: BuildState;
16         onDismiss: () => void;
17     }

```

```

18
19     export function CompileResult({ build, onDismiss }: Props) {
20         const { t } = useTranslation();
21         if (build.status === "idle" || build.status === "building") return null;
22
23         return (

```

```

24     <section
25       data-testid="compile-result"
26       className={
27         "mt-6 rounded-xl border p-4 " +
28         (build.status === "error" ? "border-red-200 bg-red-50" : "border-green/30 bg-
green/5")
29       }
30     >
31       <div className="flex items-start justify-between gap-3">
32         <h2 className="font-semibold text-ink">
33           {build.status === "done" ? t("result.ready") : t("result.failed")}
34         </h2>
35         <button
36           type="button"
37           onClick={onDismiss}
38           aria-label={t("result.dismiss")}
39           className="rounded-full px-2 text-ink/50 hover:text-ink"
40         >
41           ?
42         </button>
43       </div>
44
45       {build.status === "done" ? (
46         <div className="mt-3">
47           {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
48           <video
49             data-testid="reel-video"
50             src={build.url}
51             controls
52             playsInline
53             className="w-full max-w-xl rounded-lg bg-black"
54           />
55           <a
56             href={build.url}
57             download={build.filename}
58             className="mt-3 inline-block rounded-full bg-green px-4 py-2 text-sm font-
semibold text-[#04140D]"
59           >
60             {t("result.download", { filename: build.filename })}
61           </a>
62         </div>
63       ) : (
64         <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
65           {build.error}
66         </p>
67       )}
68     </section>
69   );
70 }
71

```

39. assistant (2026-06-21T07:41:02.970Z)

I have everything I need. The `` element confirms the reel download mechanism. Gemini requires `GEMINI\_API\_KEY` (raises if absent). The stub runner is the CPU-only real-pipeline path. `/api/health` now exists (main.py:126). The SPA builds the download URL itself via `highlightUrl()` and does NOT consume `download\_url`.

Let me do one final check on the i18n setup (the Hindi half of the goal) and the LanguageSwitcher to confirm the multilingual workflow is real.

40. user (2026-06-21T07:41:06.650Z)

=== LanguageSwitcher ===

```

import { useTranslation } from "react-i18next";
import { SUPPORTED_LOCALES, DEFAULT_LOCALE, type Locale } from "../i18n";

// Each locale's label is its own autonym, shown in its own script. Kannada renders crisply once
// the
// bundled Noto Sans Kannada font lands (tracker B4); most devices already ship a system Kannada
// font.
const LANG_LABEL_KEY: Record<Locale, string> = {
  en: "lang.en",
  kn: "lang.kn",
  hi: "lang.hi",
};

/**
 * A persistent language switcher. Selecting a locale calls i18next's changeLanguage, which the
 * detector caches to localStorage (LANG_STORAGE_KEY) ? so the choice is the top-priority signal
on
 * the next visit (negotiation: persisted pref wins). Icon-free + non-color active state via
 * aria-pressed keeps it usable for non-tech-savvy users (FRIENDLY_AND_MULTILINGUAL §4).
 */
export function LanguageSwitcher() {
  const { t, i18n } = useTranslation();
  // Highlight the user's CHOSEN locale (i18n.language), not the resolved-for-content one:
  // resolvedLanguage falls back to `en` when a catalog is missing, so picking Kannada (content
  // still English until the kn catalog lands, B5) must still show Kannada as selected. Compare
on
  // the base subtag so a detected "en-IN" highlights "en".
  const active = (i18n.language || DEFAULT_LOCALE).split("-")[0];

  return (
    <nav aria-label={t("lang.label")} className="flex items-center gap-1">
      {SUPPORTED_LOCALES.map((loc) => {
        const isActive = active === loc;
        return (
          <button
            key={loc}
            type="button"
            lang={loc}
            aria-pressed={isActive}
            onClick={() => void i18n.changeLanguage(loc)}
            className={[
              "rounded-full px-3 py-1.5 text-sm font-semibold transition",
              isActive ? "bg-white text-ink" : "text-white/70 hover:text-white",
            ].join(" ")}
          >
            {t(LANG_LABEL_KEY[loc])}
          </button>
        );
      })}
    </nav>
  );
}

=== i18n index (langs) ===
3:// Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4://   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
8://   - always-fallback to `en` so a missing translation degrades to English, never a raw key.
13:import enCommon from "../locales/en/common.json";
14:import knCommon from "../locales/kn/common.json";
15:import hiCommon from "../locales/hi/common.json";
17:// Supported locales (D6): `en` + `kn` + `hi`. Anything else falls back to `en`.
18:export const SUPPORTED_LOCALES = ["en", "kn", "hi"] as const;
20:export const DEFAULT_LOCALE: Locale = "en";
22:// ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker B3),
// so a
26:// Catalogs are co-located here (D9). `kn` + `hi` shipped (B5/B6, AI-drafted ? native review

```

```

27:// pending). A missing key in any locale still falls back to `en` ? the by-design invariant.
28:const resources = {
29:  en: { common: enCommon },
30:  kn: { common: knCommon },
31:  hi: { common: hiCommon },
32:  resources,
33:  fallbackLng: DEFAULT_LOCALE,
34:  supportedLngs: [...SUPPORTED_LOCALES],
35:  nonExplicitSupportedLngs: true, // en-IN -> en
36:  // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
37:  order: ["localStorage", "querystring", "navigator"],
38:  caches: ["localStorage"],
39:  react: { useSuspense: false },
40:// Keep <html lang> in sync with the active locale (screen readers + SEO use it). Reflects
41:the
42:=== hi common.json has the studio keys? ===
43:
44:hi keys sample:
45:{
46:  "app": {
47:    "badge": "?????-???? ??? . ??????",
48:    "title": "???? ??? ?? ??? ??????? ??????",
49:    "intro": "<b>???? ?? ?????</b> ?? ??? ?? ??????? ???? ? ??? <i>????? ???? ??? 15?</i> ?
50:??? ????? ??? ???? ??????? ?? ????????? ?????? ?? ??? <b>?????</b> ???? ???? ??; ?? ?? ???? ?????
51:????? ??????"
52:  },
53:  "lang": {
54:    "label": "????",
55:    "en": "English",
56:    "kn": "????",
57:    "hi": "?????"
58:  },
59:  "bulk": {
60:    "selectAll": "??? ??????",
61:    "clear": "???? ??????",
62:    "invert": "?????"
63:  },
64:  "mode": {
65:    "demo": "???? ???? ? ???? ???? ??? ???? ?? ??? ???? ???? ?? ???
66:<code>?video=?video_id?</code> ?? ??????",
67:    "loading": "<code>{videoId}</code> ?? ??? ??????? ???? ?? ??? ??????",

```

41. assistant (2026-06-21T07:42:46.013Z)

I have a complete, evidence-grounded picture. The Hindi catalog is fully populated and the LanguageSwitcher is wired with localStorage persistence and `?lang=` querystring negotiation. Here is the full report.

Integration Surface Map: "GCS-bucket video ? UI ? downloaded reel" (+ Hindi repeat)

****Repos / branches read this session:**** SPA `khehsutra/web/` (on `master`, up to date); engine `badminton-highlight-indexer/` on branch `feat/serving-m1-deployment-profile` (the "M1" branch; remote ref reported `gone`). Two design docs (`LOCAL\_APPLIANCE\_ARCHITECTURE.md` DRAFT, `PER\_RALLY\_SELECTION.md` IN PROGRESS/M0-complete) were authored ****this same session**** against this exact source and are heavily `[verified]`-tagged ? my findings corroborate them and add a few corrections (notably `api/health` and `download\_url` now ****exist****, which those docs list as missing/pending).

1. Engine API inventory

All endpoints are in `backend/main.py` unless noted. Detector wiring is via the segmenter registry; the API itself is detector-agnostic.

```
| Endpoint | Method | File:line | What it does | Real vs stub |
|---|---|---|---|---|
| `/` | GET | `main.py:53` | Serves `backend/api/static/index.html` (the legacy static UI,
**not** the React SPA) | n/a |
| `/api/config` | GET | `main.py:122` | Returns the whole typed `global_config` (SPA reads
`stitching.min_shot_count`) | real |
| `/api/health` | GET | `main.py:126` | Liveness probe: `{status, sport, default_segmenter}`.
**NOW EXISTS** (design docs say "expected but does not exist" ? stale) | real |
| `/api/process` | POST ? 202 | `main.py:254` | Submit a job. Fast-fails on bad path / missing
file / unknown segmenter, then enqueues; returns `{job_id, status:"queued", poll}`. Slow work
(incl. MD5 hashing + segmentation) runs on the worker thread | real pipeline |
| `/api/jobs/{job_id}` | GET | `main.py:288` | Job state `{status, progress, video_id, error,
result, reports}`. `result.video_id` is how a finished job hands a `video_id` to the picker |
real |
| `/api/query` | POST | `main.py:345` | List a video's rallies ? `{play_segments:[...]}`. **The
only rally-list path.** | real (reads DB) |
| `/api/compile` | POST | `main.py:356` | Stitch `segment_ids` ? one MP4; applies the silent
`min_shot_count` floor (`386`); returns `{status, filepath, download_url}` | real |
| `/api/highlights/{video_id}/{filename}` | GET | `main.py:321` | `FileResponse` streaming the
compiled reel from `output_dir` (path-traversal-guarded). **This is the browser-reachable
download endpoint** | real |
| `/api/upload/init` | POST | `uploads.py:64` | Start chunked upload ? `{upload_id, max_part_mb,
next_part}` | real |
| `/api/upload/{id}/part/{i}` | PUT | `uploads.py:89` | Append one sequential raw-bytes part
(per-part + total caps) | real |
| `/api/upload/{id}/complete` | POST | `uploads.py:123` | Assemble ? `{path, video_id, ...}`
ready for `/api/process` | real |
| `/api/upload/{id}` | DELETE | `uploads.py:147` | Abort + delete partial | real |

**Absent (confirmed by the design docs, not re-grepped exhaustively):** `GET /api/capabilities`,
`GET /api/videos` (only single-row `get_video(id)`), `GET /api/reels/{video_id}`, any
source/clip/thumbnail streaming endpoint, any Dockerfile, any cost ledger.

**Worker (not an HTTP endpoint):** `python -m backend.worker {infer|train}`
(`backend/worker/__main__.py`) is a **separate CLI dispatcher**, not reachable from the FastAPI
loop. It is the only thing that speaks bucket URIs today (see Q2).

---
```

2. Bucket ingest ? can the engine process a GCS video today?

****Yes, but only via the worker CLI ? not via the HTTP API.****

```
- **GCS is a first-class, fully-implemented `StorageBackend`** (`backend/storage/gcs.py`):
`fetch_to_local` downloads a blob (`30-39`), `put` uploads (`41-43`), plus
`list`/`exists`/`stat`/`signed_url`. It lazy-imports `google-cloud-storage` (`24`) and uses
**Application Default Credentials** (`GOOGLE_APPLICATION_CREDENTIALS` SA-JSON or workload
identity ? never config; `gcs.py` module docstring `1-6`).
- **URI routing:** `parse_uri` (`backend/storage/ref.py:48-68`) recognizes `gcs://bucket/key`
and normalizes `gs://` ? `gcs://` (`60-61`). `__init__.py::fetch/put/list_uris` (`56-77`)
dispatch on the scheme. Config `StorageConfig` (`models.py:295-319`) supports `backend:
Literal["local","s3","gcs","gdrive"]` (default `"local"`) with a per-backend `gcs: dict`
settings slot (`318`).
- **The worker `infer` path does the pull ? pipeline ? push** (`worker/__main__.py:cmd_infer`):
  - `101` ? `storage.fetch(args.video_uri, scratch/in.mp4)` downloads the GCS blob to local
scratch.
  - `105` ? computes the `video_id` (MD5 of the pulled bytes).
  - `130` ? runs the configured segmenter (same registry as the API).
  - `160-175` ? serializes `outputs/<video_id>/{intervals.csv,report.json}` and **pushes them
back** to `--out-uri` (which can itself be `gcs://`).
```

```
CLI: `python -m backend.worker infer --video-uri gcs://bucket/match.mp4 --out-uri
gcs://bucket/out [--segmenter ...] [--set k=v]`.
```

****Critical gap for the CUJ:**** the worker writes `intervals.csv` + `report.json` to a bucket ? it does ****NOT**** write to the API's SQLite `play\_segments` table, and it does ****NOT**** create a `videos` row that `/api/query` / `/api/compile` can read. So a worker `infer` run alone does ****not**** give the SPA a queryable `video\_id`. Conversely, `POST /api/process` only accepts a ****local**** `video\_path` (`ProcessRequest.video\_path`, validated by `validate\_input\_path` + `os.path.exists` at `main.py:264-269`) ? it has ****no bucket-URI ingest****. There is no HTTP "ingest from `gcs://`" endpoint. This is exactly the "ingest-from-bucket wiring is ****net-new****" gap the appliance doc flags (`LOCAL\_APPLIANCE\_ARCHITECTURE.md` §4.2 row "Bucket URI" = ****design****, and the reserved "bucket-worker bridge" T14).

****Other backends:**** `s3` (AWS/R2/D0-Spaces/MinIO via `endpoint\_url`) is fully implemented (`s3.py`); `gdrive` (`gdrive.py`) reads a ****locally-mounted**** Google Drive as plain files ? usable but not a remote API client.

3. Detectors / compute ? what a real dry-run needs

Segmenters registered (`segmenters/\_\_init\_\_.py`, `list\_available()`): `motion`, `gemini`, `yolo\_hybrid`, `tracknet\_hybrid`, `wasb\_hybrid`, `fusion\_hybrid`.

- ****`config.json` default = `gemini`**** (root `config.json` `indexing.default\_segmenter: "gemini"`). (Note: the Pydantic *code* default is `yolo\_hybrid` at `models.py:126`, but the shipped `config.json` overrides it to `gemini`.)
- ****Gemini path (cloud, real):**** `gemini.py:86-93` ****requires `GEMINI\_API\_KEY`**** in the environment and raises a clear error if absent. Runs on CPU (uploads downsized chunks to Google). ****Needs: a Gemini API key + network. No GPU, no WSL.**** This is the realistic CPU dry-run detector.
- ****Native WASB GPU path:**** `wasb\_hybrid` ? `TrajectoryHybridSegmenter.\_resolve\_runner` (`trajectory\_hybrid.py:65-86`). `detector\_impl` ? `{auto=WSL, native=Linux-native GPU, stub=CPU mock}` (default "auto", `models.py:133`; `RALLY\_DETECTOR\_IMPL` env overrides). ****Needs a GPU host**** (+ WSL for `auto`, or a native Linux GPU box for `native`).
- ****`trajectory\_hybrid.\_build\_native\_runner` still ****raises NotImplementedError**** (`:57-63`) ? only `wasb\_hybrid` overrides it with a real `NativeWasbRunner` (`wasb\_hybrid.py:40-46`). Native is ****owner-byte-parity-gated, not production-verified**** (per `LOCAL\_APPLIANCE\_ARCHITECTURE.md` §3, risk table).**
- ****`motion` (CPU, real but low-quality):**** `motion.py` ? pure OpenCV frame-differencing, ****no key, no GPU****. BUT it ****fabricates**** `server`/`receiver` (`random.sample`), `shot\_count`, and `events` (`motion.py:191-239`) ? which the SPA is explicitly told never to surface (`types.ts:20-26`, `PER\_RALLY\_SELECTION.md` §3 "Fabricated ? do NOT surface"). It produces real *segment boundaries* though.
- ****`stub` (CPU mock):**** `detectors/stub\_runner.py` ? deterministic synthetic trajectories, ****no GPU/WSL/weights****. Combined with `--set indexing.skip\_ai\_handoff=true` it is the validated ****secret-free, GPU-free**** path (`worker/\_\_main\_\_.py:9-11`). Output is ****not real detection**** ("mock ? not real detection" per the doc's honesty matrix).

****CPU-only paths that produce real-ish rallies:**** `gemini` (real, needs key) and `motion` (real boundaries, fabricated metadata). `stub` is mock. ****For a true "real rallies, no GPU" dry-run, Gemini + a key is the path.**** A GPU is only required for native/WSL WASB.

4. Reel download to the local machine

****Fully working today.**** Two-step, browser-reachable:

1. `POST /api/compile` writes the MP4 into `output\_dir` and returns BOTH a server-local `filepath` ****and**** `download\_url` (`main.py:410-411`): `download\_url = f"/api/highlights/{quote(video\_id)}/{quote(out\_name)}"`. ****The `download\_url` field exists**** (the "M1" addition ? `PER\_RALLY\_SELECTION.md` P6 lists it as `?`/TODO, but it is in the code now).

2. ``GET /api/highlights/{video_id}/{filename}`` (``main.py:321-342``) streams the file via ``FileResponse`` with the right media type ? the browser-hittable endpoint.

SPA side: the SPA does **not** consume ``download_url``. Its ``CompileResponse`` type (``types.ts:33-37``) only declares ``status`` + ``filepath``; after compile it **hand-builds** the URL itself via ``highlightUrl(videoId, filename)`` (``client.ts:86-88``, called in ``App.tsx:133``), then renders an inline ``<video src>`` preview + an ``<a href download>`` link (``CompileResult.tsx:48-61``). So download-to-local **works end to end already** for any video that's in the engine's DB ? the only redundancy is the SPA ignoring the server-provided ``download_url``.

5. The SPA gap ? what's missing for this CUJ

The SPA is **strictly job-tethered and read-only on ingest**. Hard evidence: a grep of ``web/src`` for ``process|upload|jobs|capabilities|videos|reels|health|download_url|job_id`` returned **zero matches**. The SPA's entire API surface is ``client.ts``: ``listRallies`` (``/query``), ``compileReel`` (``/compile``), ``highlightUrl`` (``/highlights``), ``getConfig`` (``/config``). That's it.

Concretely missing:

- **No ingest/process UI at all.** The SPA cannot point at a video (bucket or local), cannot call ``/api/process``, cannot call the ``/api/upload/*`` chunked endpoints, cannot pick a segmenter. It assumes a video is **already processed**.
- **How it gets a ``video_id``:** purely from the URL query string ? ``?video=<id>`` (``App.tsx:48``: ``new URLSearchParams(...).get("video")``). With no ``?video=`,` it shows hardcoded ``DEMO_RALLIES`` (``App.tsx:30-37``). There is **no in-UI path** to obtain a ``video_id`` ? no job submission, no poll of ``/api/jobs/{id}``, no ``/api/videos`` list.
- **No progress/poll UI** (no ``/api/jobs`` polling ? that's appliance doc screen S2, unbuilt).
- **No history/reload survival** (no ``/api/videos`` / ``/api/reels`` ? S4, unbuilt; the endpoints don't even exist).

What the SPA **does** do well (M0 complete): load ``?video=<id>`` ? ``POST /api/query`` (default all-selected) ? deterministic query/select/bulk ? ``POST /api/compile`` ? ``<video>`` preview + download, with ``min_shot_count`` warning from ``/api/config``, structured ``cid``-traced logs, and full i18n (en/hi/kn). So the **back half** of the CUJ (review ? select ? compile ? download) is built; the **front half** (point at a bucket video ? process ? get a ``video_id``) is entirely missing from the UI.

6. Cheapest path to a dry run

The minimum to demonstrate "GCS video ? UI ? downloaded reel" **without building in-UI ingest** is to pre-seed the engine DB out-of-band, then drive the existing SPA back-half against it. The blocker is that **neither** the worker ``infer`` (writes CSV to bucket, not the DB) **nor** ``/api/process`` (local path only) bridges "GCS video ? a queryable ``video_id`` in the API's SQLite". So the cheap path is:

Steps (which already work vs must be run/built):

1. **Get the bucket video onto the engine host as a local file** ? ``python -m backend.worker infer --video-uri gcs://bucket/match.mp4 --out-uri gcs://bucket/out --set indexing.skip_ai_handoff=true`` *fetches* it to scratch, OR simpler: ``gsutil cp gcs://bucket/match.mp4 ./incoming/`` (the ``gcs.py`` fetch already proves GCS read works). ? *fetch works; trivial to run.*
2. **Process it through the HTTP API so it lands in the API's DB** ? ``POST /api/process {video_path: "<abs local path>", segmenter: "gemini"}`` (needs ``GEMINI_API_KEY``) ? poll ``GET /api/jobs/{job_id}`` until ``result.video_id`` appears. This populates ``videos`` + ``play_segments`` that ``/api/query`` and ``/api/compile`` read. ? *endpoint works; just needs a key + a real video.* (Or ``segmenter: "motion"`` for a key-free real-boundaries run, accepting fabricated-metadata caveats.)
3. **Open the SPA pointed at that video** ? ``https://<host>/?video=<video_id>`` (dev: ``npm run``

dev`, Vite proxies `/api` ? `:8000`). The SPA auto-loads rallies via `POST /api/query`. ? *works today.*

4. **Select ? Build ? download** ? pick rallies, click Build ? `POST /api/compile` ? `` preview + `` saves the MP4 locally. ? *works today.*

5. **Repeat in Hindi** ? click the `????` button in `LanguageSwitcher` (or append `?lang=hi`, or set localStorage), re-run select?compile?download. ? *works today* (Hindi catalog fully populated, `i18n/index.ts` + `locales/hi/common.json`).

So the only genuinely missing glue is step 2's "point at GCS" being a CLI/manual step, not in the UI. If you accept a 1-command pre-process to mint the `video\_id`, the dry run needs **zero new code** ? only a running engine with a Gemini key, the GCS video copied/fetched local, and the SPA opened with `?video=<id>`.

The alternative (full in-UI ingest) would require building appliance-doc screens S1/S2 + the **net-new bucket-ingest HTTP endpoint** (T12/T14) ? much more work and explicitly out of the current MVP scope.

7. Dependencies / unknowns to decide or stand up

- **Engine host & OS-runnability:** The engine binds `127.0.0.1:8000` (`main.py:655`) and needs `ffmpeg` + OpenCV. It's pure-Python FastAPI and **should** run on Windows for the **Gemini/motion** CPU paths (no WSL needed). **WSL/Linux-GPU** is required only for `wasb` native/auto detection. The validated GPU-free e2e is documented on the **CPU DigitalOcean droplet** (`168.144.126.176`, blr1, CPU-only). Decide: run the dry-run engine on **local Windows** (Gemini-CPU, simplest), the **CPU droplet**, or a **GPU host** (only if you want real WASB). For the bucket dry-run, local Windows + Gemini is the fewest moving parts.

- **Compute choice:** Gemini-CPU (real, needs key, cloud upload ? consent copy) vs `motion` (key-free, fabricated metadata hidden) vs GPU/WASB (owner-byte-parity-gated, not production-verified). For "real-ish rallies, cheapest" ? **Gemini + key**.

- **Which bucket video:** must pass the engine's validators (`config.json` validation: ?1280x720, ?29.9 fps, blur/scene-cut/court-relevance checks) or `/api/process` returns `status:"failed"`. Pick a clean ?720p badminton clip.

- **GCS auth:** `GOOGLE\_APPLICATION\_CREDENTIALS` ? SA-JSON (or `gcloud` ADC). Never in config (`gcs.py` docstring). Only needed if you use the worker fetch rather than `gsutil`.

- **CORS / SPA?engine connection:** engine sets CORS `allow\_origins=[""]`, `allow\_credentials=False` (`main.py:41-47`) ? so cross-origin browser calls are permitted. In dev, Vite **proxies** `/api` ? `localhost:8000` (`vite.config.ts`), so it's same-origin and CORS is moot. In prod the appliance design is a single reverse-proxy origin (the gate, not the app, is the boundary). For a quick dry-run, `npm run dev` against a local engine needs no CORS/auth work.

- **Edge auth:** none in the engine by design (`LOCAL\_APPLIANCE\_ARCHITECTURE.md` §4.4 ? Caddy basicauth or cloudflared+Access). Not required for a localhost dry-run; **required** before any non-localhost exposure (and **rotate** the chat-shared `GEMINI\_API\_KEY`).

- **The DB-bridge unknown (the real blocker):** there is **no supported in-API way** to ingest a `gcs://` URI into a queryable `video\_id`. For the dry-run this is sidestepped by the local-copy + `/api/process` pre-step (Q6); productizing "bucket video from the UI" needs the net-new bucket-ingest endpoint (T12/T14) ? an owner-gated decision.

Honest flags

- The two khelsutra design docs are `[design]`/[verified]` planning docs, **not implemented appliance code** ? `LOCAL_APPLIANCE_ARCHITECTURE.md` is `DRAFT` with an all-`?` tracker; only `PER_RALLY_SELECTION.md` M0 (the SPA pick?reel back-half) is actually shipped.

- Docs claim `/api/health` is missing and `download\_url` is a TODO ? **both are now present** in the engine code on this branch (`main.py:126`, `main.py:411`). The SPA still doesn't consume `download\_url`, and `/api/health` has no SPA caller.

- **native** WASB is partially wired and owner-parity-gated; treat as **not production-verified**. `stub` is explicitly mock. `motion`'s `server/receiver/events` are fabricated and must stay hidden.

- I did not run anything (read-only mapping). The "works today" claims for the SPA back-half rest on the M0 changelog's stub-engine e2e verification plus the code I read, not a live run in

this session.

```
**Key files:** SPA ? `web/src/api/client.ts`, `web/src/api/types.ts`, `web/src/App.tsx`,  
`web/src/components/CompileResult.tsx`, `web/src/components/LanguageSwitcher.tsx`,  
`web/src/i18n/index.ts`, `web/vite.config.ts`. Engine ? `backend/main.py`,  
`backend/api/uploads.py`, `backend/worker/__main__.py`,  
`backend/storage/{gcs.py, local.py, ref.py, __init__.py, base.py}`, `backend/config/models.py`, `bac  
kend/pipeline/segmenters/{__init__.py, gemini.py, motion.py, wasb_hybrid.py, trajectory_hybrid.py}`,  
`backend/pipeline/detectors/stub_runner.py`, `config.json`. Docs ?  
`kheIsutra/docs/{LOCAL_APPLIANCE_ARCHITECTURE.md, PER_RALLY_SELECTION.md}`.
```